

SECOND EDITION

THE

PS2

LESSONS
LEARNED
(SO FAR)

PROGRAMMING
LANGUAGE

NINJA DYNAMICS
JUNE 2026

Contents

Chapter 1 - The PS2 Mental Model

Chapter 2 - Source Boundaries and Port Architecture

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Chapter 4 - Toolchain and Build-System Traps

Chapter 5 - GS Display, Framebuffers, and Video Modes

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Chapter 8 - Depth, Blending, and 2D Primitives

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Chapter 10 - Texture Formats, Alpha, and Filtering

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Chapter 12 - GS and EE Memory Management

Chapter 13 - Procedural Effects Under GS Constraints

Chapter 14 - Input, Debug UI, and Save Data

Chapter 15 - SPU2 SFX Fundamentals

Chapter 16 - Music Streaming and Custom IOP Modules

Chapter 17 - Validation Doctrine

Chapter 18 - Extending ps2gl and ps2stuff at Ownership Boundaries

Chapter 19 - Delivering Vertex Fog to the GS

Chapter 20 - Interrupt Completion, Vertex Alpha, and Batched Submission

Chapter 21 - Coexisting Mipmaps and the Long-Run Reset

Chapter 22 - Writing VU1 Microcode Through VCL: The Clip Renderer

Chapter 23 - Separating Preparation Cost from Submission Cost

Chapter 24 - The Heap Smasher: Forensics When the Crash Site Lies

Chapter 25 - GS Context Reuse and Mipmap Page Locality

Chapter 26 - Submission LOD, Texture Identity, and Semantic Vertex Channels

Chapter 27 - Split Immutable Models at the Dynamic Material Boundary

Chapter 28 - Audio Ring Geometry Is a Wire ABI

Chapter 29 - What Each Kind of Validation Can Prove

Chapter 30 - Construction, Publication, and Completion

Chapter 31 - Compact Geometry Without Changing Its Meaning

Chapter 32 - Emitted Code and Cache Dependencies

Chapter 33 - Mip Layout, Dither, and Depth

Chapter 34 - Alignment Is Part of the Asset Format

Chapter 35 - Profile the Pipeline Without Measuring the Instrument

Appendix A - Quick Rules

Appendix B - Case Study: HyperSolar's PS2 Port

The PS2 Programming Language

Lessons Learned (So Far)

by Ninja Dynamics

Public Release Edition, June 2026

Technical updates through September 25, 2026.

This handbook teaches PS2 programming contracts: address spaces, DMA ownership, GS state and precision, IOP services, and validation on real hardware. Project milestones, asset policies, and selected runtime defaults belong in project documentation, not in the rules of this book.

This handbook is distilled from HyperSolar's PS2 work. It is written for the person who already has the SDK, already has the manuals, and is now staring at a black screen, a silent SPU2, a wrong-colored CLUT, or a render path that works in PCSX2 and fails on hardware.

It is not a replacement for Sony's manuals. It is the scar tissue layer: the rules that only became obvious after ps2gl, the GS, SPU2, SIF RPC, PCSX2, ps2link, and real hardware disagreed with one another.

The promise is practical: every chapter should leave you with a rule you can use in a real game, a failure mode you can recognize, and a reference you can open when you need to go deeper.

Public release note: this edition is written to stand alone. Implementation names, helper names, and subsystem names appear as concrete examples, not as dependencies the reader must have.

Why does this exist

Most of what is hard about the PS2 is not in any one manual. The Sony manuals are excellent first-principles references — they tell you exactly what every GS register does and exactly how an SPU2 voice is structured. What they cannot tell you is which of those facts will quietly ruin your week: that an incorrectly reserved color/Z layout can render correctly in an emulator and corrupt on hardware; that `oglFinish()` destroys your GL context instead of flushing it; that a buffer-loaded IRX can report success and never register its RPC server. Those are emergent properties of the *combination* — EE plus GS plus IOP plus SPU2 plus a forked OpenGL-ish library plus an emulator plus a network loader — and they only show up when the pieces disagree.

This handbook is the record of those disagreements and how they were resolved. It is deliberately opinionated: where a rule has survived real hardware validation it is stated plainly, and where a fix is empirical-but-unproven it says so.

How to read this handbook

Each chapter is self-contained and built around the same spine: a rule you can apply, the failure mode that makes you reach for it, and a pointer to the Sony manual chapter that explains the underlying hardware. Read straight through for a tour of the platform and its software stack, or jump to the chapter that matches your current black screen. Chapter 1 is the mental model everything else assumes;

Appendix A is the whole book compressed to a checklist; Appendix B collects the historical context of the worked examples.

Conventions

- **"Validate on hardware"** appears often and is not boilerplate. PCSX2 is a different witness from a real console (see Chapter 17). When a rule is hardware-only, the text says which class of bug the emulator hides.
- **Manual citations** name the Sony manual and the relevant chapter so you know where to open for first-principles detail. The GS User's Manual splits into *Local Memory* (Ch 2, pixel-storage modes and page geometry), *Drawing Function* (Ch 3, texture mapping, alpha-blending, dithering, pixel test), *Image Data Transmission* (Ch 4, EE↔GS transfers and GIF), *CRTC* (Ch 5, display/video output), and *Registers* (Ch 7) — those splits are referenced by number throughout.
- **Confidence is marked.** Proven rules are stated as rules; inferred ones are flagged as empirical so you do not inherit a guess as a fact.

Sources and reference shelf:

- HyperSolar PS2 development notes, distilled and rewritten for public release.
- Ninja Dynamics, ps2gl and ps2stuff Fork Delta Report, June 2026.
- Sony PlayStation 2 GS User's Manual: GS registers, display environment, pixel storage modes, CLUTs, texture registers, alpha blending, scissor, GIF packet formats, and image transfers.
- Sony PlayStation 2 GS User's Manual Supplement: GS errata and corner cases.
- Sony PlayStation 2 EE Overview Manual: EE, VU0/VU1, GIF, VIF, DMAC, IPU, and the system architecture.
- Sony PlayStation 2 EE User's Manual: DMAC, GIF, VIF, timers, INTC, scratchpad, and transfer behavior.
- Sony PlayStation 2 EE Core User's Manual: R5900 CPU, caches, MMU, COP0, and pipeline behavior.
- Sony PlayStation 2 EE Core Instruction Set Manual: R5900 and MMI instructions.
- Sony PlayStation 2 VU User's Manual and VU Instruction Manual: VU architecture, micro-mode, and microinstructions.
- Sony PlayStation 2 SPU2 Overview Manual: two cores, 24 ADPCM voices per core, 2 MB SPU2 RAM, voice registers, PITCH, ADSR, KON/KOFF, VMIX, and mixing.
- Community PS2 texture-format references are useful quick companions to the GS manual's PSM and CLUT tables.

Manual citations in this handbook name the relevant Sony manual by title. No quoted manual text is required to use this handbook; the manual references are there so you know which hardware chapter to open when you want first-principles detail.

Chapter 1 - The PS2 Mental Model

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) and Ch 3 (Functional Overview) for how the EE core, VU0/VU1, GIF, VIF, DMAC, and IPU connect and feed the GS; GS User's Manual Ch 1 (Overview) for the GS block configuration; SPU2 Overview Manual Ch 1 (Overview of SPU2) for the audio side.

Before any specific rule, internalize the shape of the machine — almost every trap later in the handbook is really a consequence of getting one of these seams wrong (an EE pointer handed to the IOP, a stream forced through a DMA boundary, an emulator standing in for a CPU it does not fully model).

The PS2 is several machines tied together by DMA and RPC-like boundaries. Treat those boundaries as real architectural seams.

The EE runs the game, simulation, asset orchestration, and most rendering glue. The VUs can handle transform and render submission; libraries such as ps2gl provide supported paths. An EE software clipper remains useful for geometry that a chosen VU renderer cannot safely clip. The GS is not a modern forgiving GPU; it is a rasterizer with strict VRAM page geometry, fixed-point coordinate limits, explicit texture registers, and very sticky state.

The IOP owns many services that look "system level" from the EE: modules, filesystems, memory cards, libsd/ps2snd, USB mass storage, and custom IRX code. The EE speaks to it through SIF RPC and DMA. If an API takes a pointer and the work happens on the IOP, ask which CPU's address space the pointer belongs to.

The SPU2 is a local-memory ADPCM synth, not a stream-player. It gives excellent per-voice pitch/volume control once samples are resident in SPU2 RAM. Long music requires streaming into SPU2 memory while the voice plays.

PC, PCSX2, ps2link, and real hardware are four different witnesses. PCSX2 is great for correctness and crashes, but it can hide GS display-register bugs, texture precision artifacts, mip blend quantization, and some hardware-only filtering behavior. ps2link can mask standalone IOP boot problems because it leaves the IOP in a friendlier state than a disc, USB boot, or emulator launch. Real hardware remains the final judge.

Chapter 2 - Source Boundaries and Port Architecture

Relevant manuals: mostly architectural — EE Overview Manual Ch 1 (Architecture Policy) frames why the EE/GS/IOP division of labor exists in the first place. This chapter is otherwise engineering discipline drawn from the case study, not hardware behavior.

This chapter is about a different kind of correctness than the rest of the handbook: not "what does the hardware do" but "where does each responsibility live in your code." It earns its place because every PS2-specific bug is cheaper to fix when the platform code is a thin, parameter-fed library than when it has quietly grown into a second copy of the game with its own drifting scene order.

Keep hardware and service mechanisms separate from application policy. A PS2 backend can expose rendering and I/O primitives while shared application code supplies resources, transforms, ordering and gameplay-derived parameters.

It may own:

- pad adapter code;
- embedded asset registry callbacks;
- GS-safe texture helpers;
- alpha conversion and blend-state helpers;
- the software clipper;
- display-list compile/call helpers;
- PS2 draw primitives with explicit input, state and lifetime contracts;
- scene resource installers that turn already-loaded assets into PS2 helper meshes, display lists, or baked textures;
- memory-card blob I/O and low-level PS2 service code.

It should not:

- read application globals; the pad adapter returns input through an explicit interface;
- name game asset files directly;
- compose scenes;
- own stage sequencing, gameplay state transitions, timers, scoring, or sim behavior.

Every value derived from game state crosses into the PS2 helper as a parameter: camera, anchor, floor type, scroll values, model transforms, lighting, tint, bounding sphere, sun position, or effect intensity. This keeps the cross-platform files as the source of truth.

The scene graph should remain shared. In the case study, duplicated PS2 scene code drifted in draw order. That drift was not cosmetic: haze drew after particles and made explosions look behind fog. Keep new scene elements in the shared scene function and put PS2-specific behavior at leaf helpers.

When porting a render function, identify every read of application state and make it an explicit input or keep it in shared scene code. Hardcoding one material composition into a platform helper can silently change another scene.

Resource lifetime should follow the resource's actual use. Load and install scene-dependent textures through the application resource manager; release their PS2 representations when no remaining

consumer needs them. Mipped textures need teardown for all hidden levels and any shared allocation owner, not just their public base handle.

The backend API has two classes of calls:

- public one-shot draw calls and public `*_begin()` calls must fence their own GS state;
- hot-loop `*_add()` or `*_draw()` calls between begin/end should be lean and inherit state from the matching begin.

For example, an overlay helper that restores generic state may re-enable culling. A subsequent mesh draw must establish the winding and cull convention it needs, even if that convention differs from the preceding mesh.

Chapter 3 - Boot, IOP Bring-Up, and Standalone Reality

Relevant manuals: EE Overview Manual Ch 2 (Architecture Overview) for the EE/IOP/SIF relationship; EE User's Manual for the DMAC and SIF transfer paths the RPC layer rides on. For audio-side consequences, see SPU2 Overview Manual Ch 1.

The IOP is the part of the machine most likely to lie to you at boot. It will accept a module, hand you back a positive id, and quietly never finish bringing it up — and the one tool you would normally use to see this (`printf`) is gone the moment you leave the network loader. This chapter is the bring-up ritual and the breadcrumb techniques that make a silent IOP visible.

Do standalone IOP bring-up before loading buffer IRX modules. A module loaded with `SifExecModuleBuffer` can return a positive module id and still never register its RPC server if the IOP is in the wrong inherited state. `ps2link` masks this because its loader leaves the IOP usable; PCSX2 and real disc/USB standalone boots do not.

For a standalone application that owns IOP initialization, the case-study sequence is:

```
SifInitRpc(0);
SifIopReset("", 0);
while (!SifIopSync()) { }
SifInitRpc(0);
sbv_patch_enable_lmb();
sbv_patch_disable_prefix_check();
```

Link `-lpatches` and include the IOP control and SBV patch headers. This belongs at the top of pre-init, before memory-card modules, `libsd/ps2snd`, BDM, custom IRX modules, or any other buffer-loaded code.

`SifIopReset` drops `ps2link` console and `host: fileio`. That is acceptable only if the game no longer needs `host:` after boot. Once assets are embedded and USB or disc paths are explicit, this makes the dev run behave like the shipped standalone run, which is exactly what caught the silent-audio bug.

Do not equate "works through `ps2link` on hardware" with "works standalone." The audio stack proved the danger: `ps2snd.irx` worked over `ps2link`, but in standalone boot its `BINDID_PS2SND` RPC never came up until the IOP reset and SBV patches were added.

Make the reset build-conditional so it does not cost you the dev loop. `SifIopReset` reboots the whole IOP, and under network boot `ps2link`'s ethernet driver (SMAP) lives there — so the reset kills `ps2link`'s network. The EE program keeps running, but `ps2client reset` / `printf` die and you must physically power-cycle to reload `ps2link` before the next deploy (telltale: `ping` answers right after a hard reset, then goes unreachable the instant the ELF runs). The network-loader build can deliberately retain the loader environment if it provides the required services. Use an explicit boot-mode flag around reset / sync / re-init, keeping `SifInitRpc(0)` and the SBV patches on both paths. The standalone build stays self-contained; the network-boot dev build keeps `ps2link` alive so reset/printf survive and iteration needs no power-cycle.

When debugging standalone or PCSX2 boot hangs, `printf` is gone. `ps2sdk printf` routes to `ps2link`, and PCSX2's trace log has hardware trace categories, not a program-console stream. Use GS color breadcrumbs:

```
BeginDrawing();
ClearBackground(SOME_COLOR);
EndDrawing();
/* Complete presentation through the renderer's normal frame boundary. */
```

The screen freezes on the last completed color. PCSX2 logs can still distinguish exceptions from spins. If there is no EE exception but the log shows timeout loop skipping, suspect a bind loop or wait.

`nopdelay()` is expensive. It is about one million nops, roughly 17 ms per call. Big SDK-style loops survive only because they break on the first success. If a failure path can run to completion, a `100000 * nopdelay()` wait is effectively a hang. Use small bounded counts.

`sceSifBindRpc(cd, sid, 0)` blocks until the IOP replies. An unregistered server does not reply. For probes, use `SIF_RPC_M_NOWAIT` with persistent client storage. Before retrying, prove the previous bind idle with `sceSifCheckStatRpc`. A timeout reports failed progress but does not release RPC/DMA ownership. An unregistered SID may be dropped before the server appears; retry within a bounded deadline only after the previous transaction has completed.

Chapter 4 - Toolchain and Build-System Traps

Relevant manuals: EE Core User's Manual and EE Core Instruction Set Manual cover the R5900 the toolchain targets, but most of this chapter is GCC/binutils/ps2sdk behavior, not Sony hardware behavior.

These are the bugs that never reach the hardware because they stop at the compiler or linker — yet they cost real days, because the error messages point nowhere near the cause (a predefined macro turning a type into a number, a native-Windows assembler rejecting a POSIX path, a missing inline definition surfacing as a link error on one platform only).

The DreamSDK PS2 assembler is a native Windows executable. When MSYS invokes `mips64r5900el-ps2-elf-as.exe`, `.incbin "/c/Users/..."` fails because that path is not a Win32 path. Generate mixed Windows paths for `.incbin`, for example via `cygpath -m`, with a fallback for non-MSYS environments.

The EE compiler predefines `mips` as a macro. Do not name variables, fields, or parameters `mips`. It preprocesses to `1`, creating baffling syntax errors such as `CMMTexture* 1[6]`. Use `levels`, `mip_list`, or another name.

`RAYMATH_IMPLEMENTATION` must exist in exactly one translation unit for PS2. The PS2 toolchain plus `raylib4ps2` can fail to inline functions that other platforms resolve elsewhere, leading to undefined references such as `MatrixMultiply`. Put the implementation define in exactly one PS2-only backend translation unit.

When using forked libraries, link by explicit path and include their headers ahead of toolchain headers. In a sibling-fork layout, build `ps2stuff` as a local archive, link that archive by path, and do not install it over the toolchain copy. `ps2gl` must also compile against the forked `ps2stuff` headers if it uses fork-only APIs such as GS memory info.

Canary banners are a cheap sanity check for fork linkage. Make them print once at startup from `raylib`, `ps2gl`, and `ps2stuff`. A build-script stamp only rewrites an existing string; it cannot insert a missing `printf`.

Chapter 5 - GS Display, Framebuffers, and Video Modes

Relevant manuals: GS User's Manual Ch 5 (CRTC) for the DISPLAY/PMODE registers, magnification, and video output; Ch 2 (Local Memory) for page geometry and framebuffer storage; Ch 7 (Registers) for BGCOLOR and the privileged register addresses; GS User's Manual Supplement for errata.

This is the chapter where the emulator lies most. The GS display path — sync timing, the DISPLAY register's magnification, framebuffer page alignment, the color/Z pairing, BGCOLOR — is largely normalized away by PCSX2 and only tells the truth on a real console with a real TV. Validate display changes on the target console and output device; emulator success alone does not establish that those registers describe a valid raster.

Do not call libgraph's `graph_set_mode` or `graph_set_screen` on top of ps2gl. libgraph and ps2gl both program the GS display read-circuit registers. Calling libgraph corrupts ps2gl's display environment and can crash even in the default mode. Use `SetGsCrt` for sync-generator changes and ps2gl/fork helpers for the display registers that ps2gl owns.

`SetGsCrt` is only half of a video-mode switch. It changes scan timing, not the GS DISPLAY register's position, magnification, and size. ps2gl re-sends its display environment each frame, so a mode switch must update ps2gl's live display environment too.

The ps2sdk constant `GS_MODE_DTV_480P` is `0x50`; do not substitute the incorrect value `0x32`.

Choose visible dimensions separately from the output timing mode. For example, a 640x448 image within progressive output can reuse a 448-line allocation:

- 448 is exactly 7 rows of 64-line 16-bit GS pages.
- 480 is 7.5 rows and needs padding to 512 lines in memory.
- 448p can share the same visible `SCREEN_W/H` and texture slot map as 448i.
- A 640x448 16-bit framebuffer uses 70 pages, matching the stock 448i footprint.

GS framebuffers must be page-row aligned. A GS page is 64x32 for 32/24-bit formats and 64x64 for 16-bit formats. Do not size framebuffers by `width * height * bpp / 8192` unless the height is page aligned. For a 16-bit 640x480 buffer, 480 lines address into 7.5 page rows; the bottom-right portion spills into the next buffer unless the reservation is rounded to 512 lines.

Do not reuse a color-buffer reservation calculation blindly for Z. The case-study layouts paired 16-bit color with 16-bit Z and 32-bit color with 24-bit Z; a mixed 16-bit-color/24-bit-Z setup corrupted on hardware despite appearing correct in PCSX2. That observation does not establish a universal ban on mixed formats. Check each PSM's page geometry, base, stride and reserved range against the GS manual, then validate the complete layout on hardware.

The cost of 16-bit Z is coarser depth. Thin double-sided geometry can z-fight. Where only the exterior is intended to be visible, correct culling can remove the competing back surface. Verify each asset's winding against the renderer's front-face convention rather than prescribing one cull face for every model.

For DTV progressive, `DISPLAY.magH` must match the faster scan clock. Interlaced NTSC/PAL used `magH = 4`; DTV progressive needs `magH = 2`. Leaving the old value makes the image twice as wide on hardware. PCSX2 can normalize this away, so validate display-register work on a real console.

Runtime GS layout switching is possible, but the safe ritual matters:

- `pglFinish()` is not a flush; it destroys the GL context. Drain with `pglFinishRenderingGeometry(PGL_DONT_FORCE_IMMEDIATE_STOP)` and `pglWaitForPresentation()` before `pglWaitForVU1()` and `pglWaitForVSync()`. Retire queued geometry/presentation before destroying the slot map.
- Remove and rebuild GS memory slots, including locked framebuffer slots.
- Re-add framebuffer and Z slots, bind areas, set draw buffers, call `SetGsCrt`, set display buffers, then set the `ps2gl` video mode.
- Most textures re-upload lazily after the slot wipe, but mipmapped textures bake GS addresses into MIPTBP. Re-create them after the switch.
- Black-bracket the transition so uninitialized buffers and slot rebuilds do not scan out as VRAM garbage.
- The remaining blink is GS scan-generator resync; it is hardware behavior.

The GS `BGCOLR` register controls the margins outside the active framebuffer. `ClearBackground` does not touch it. Soft reset does not reliably clear it. Write it as a 64-bit store to physical `0x120000E0`, not a `kseg1` alias. Have the display owner maintain the intended margin color across frame and mode transitions. In a stack that overwrites it, restore it at the documented housekeeping boundary and before long stalls; otherwise the unwanted color can remain visible in the margins. This is a display-state workaround, not a replacement for finding an unexpected register writer.

The interlace flicker filter is the two-read-circuit merge, not a post pass and not `S.MODE2.FFMD`. The GS has two display read circuits (`P.MODE.EN1 / EN2`). Enable both on the *same* framebuffer, offset read circuit 1 down one scanline (`DISPFB1.DBY = 1`), and const-alpha blend them (`P.MODE.MMOD=1`, `ALP`, `SLBG=0`): a 2-tap vertical low-pass on scanout that settles interlaced field twitter (thin-line flicker, distant-edge shimmer) at zero fill-rate cost. It is the classic PS2 softening most interlaced-era titles shipped. Three facts make it work rather than ghost:

- It is meaningful only in interlaced modes. Progressive output has no field twitter, so the blend only softens the image — gate it off there.
- Both read circuits must be repointed on every buffer flip. The usual swap path updates only `DISPFB2`'s base address; with the filter on, RC1 must be repointed to the same buffer too, or the two circuits blend two different frames and the image ghosts. The one-line `DBY` offset lives in the register and survives an address-only update.
- A live raster re-center (the `DISPLAY`-only, no-border-flash path) must move `DISPLAY1` alongside `DISPLAY2`, or RC1 and RC2 pan apart into a misaligned double image.

It is a whole-scanout effect, so it softens the HUD too; that cannot be excluded without a separate render-to-texture pass. Validate the blend alpha on hardware: PCSX2 does not reproduce CRT interlace twitter, so it is a poor witness for whether a given strength helps, and RC1's `+1` line reads one scanline past the buffer on the last row.

Chapter 6 - Aspect Ratio and Full-Frame Coverage

Relevant manuals: GS User's Manual Ch 5 (CRTC) for the display environment — how the read circuit magnifies and positions the framebuffer onto the output raster, which is the root cause of the stretch this chapter corrects.

A point that surprises most newcomers: a plain 4:3 PS2 framebuffer is *not* geometrically neutral. The display hardware stretches the stored lines to fill the raster, so "render 640×448 and forget about it" ships a subtly squished world. This chapter is the correction math and — just as important — the separation between *aspect* (a projection scale) and *placement* (a HUD re-anchor), which are easy to conflate and painful to debug once you have.

The 640x448 PS2 framebuffer is not geometrically neutral on a 4:3 display. The display stretches 448 lines to the TV raster height, making the scene vertically stretched relative to square-pixel rendering. A 640x448 renderer can correct this by applying a centered horizontal scale:

```
/* Example: preserving a 640x480 square-pixel reference in 640x448. */
xscale = 480.0f / 448.0f;
if (widescreen) xscale *= 3.0f / 4.0f; /* 4:3 content on a 16:9 raster */
```

This baseline 4:3 correction and the 16:9 anamorphic squeeze use the same mechanism. The 3D pass post-multiplies projection; the 2D pass applies a matching centered transform. World-anchored 2D overlays stay aligned only if both passes use the same scale.

Aspect correction and placement are separate jobs. The pass scale corrects geometry; HUD layout must re-anchor edge positions with the inverse scale (the inverse of the centered pass transform). Centered elements need no horizontal correction.

Any pass that replaces matrices inherits both jobs. Overlay passes that load their own projection/view must re-apply the squeeze and re-anchor edge-based screen coordinates before adding half-size offsets.

"Full-screen" fills are not simply `(0,0,SCREEN_W,SCREEN_H)` inside the PS2 2D aspect pass. Horizontally, use the inverse aspect transform of both screen edges. Vertically, overscan past `SCREEN_H` because the GS fill rule can drop the exact boundary row. Let the GS scissor clip the extra pixels. The same horizontal span rule applies to tiled texture backdrops.

Chapter 7 - ps2gl State, Batching, and Frame Boundaries

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for the drawing environment and per-primitive state, and Ch 4 (Image Data Transmission) for the GIF packet formats a batch ultimately becomes; VU User's Manual for the VU1-side nature of ps2gl's transform/submit paths.

The single most useful performance instinct on this hardware: stop counting vertices and start counting state changes. ps2gl turns a render-state transition or texture bind into a re-sent GS environment packet, which dwarfs the cost of the handful of vertices it surrounds. This chapter is the batching discipline that follows from that, plus the frame-boundary rules that keep the swap path from faulting.

On ps2gl, state transitions and texture binds often cost more than vertices. Batch by state first. A tiny quad that reconfigures the full pipeline can be more expensive than dozens of vertices under one already-correct state block.

The text path proved it in stages:

- per-glyph `DrawTexturePro` was about 70 full batch setups for the gameplay HUD;
- batching each string saved several milliseconds;
- batching across strings saved more by avoiding per-string font texture binds;
- batching HUD rects was the real win, because each previous rect reset the full GL/GS pipeline for four vertices.

General rule: hot paths should use `begin/add/end`, hoist state into `begin`, and vary only uniform color or data inside.

Stock fixed-function immediate paths have restricted unlit varying-color support. Uniform-color batches or the established flat-light helper remain appropriate there. The fork also has an explicitly admitted unlit colored triangle renderer used by prepared HUD and particle paths; its finite state and attribute contract does not qualify arbitrary stock batches.

An ordered particle path can stage colored arrays while retaining bucket order and replay an equivalent fallback when a batch does not meet the fast path's contract. `r1End` alone need not construct pending geometry: flush while its bindings and admission flags are still valid.

The frame owner controls `EndDrawing`, queued presentation and scratch reuse. A historical post-`EndDrawing` timer fault motivated a narrow housekeeping boundary in one integration; it is not a universal prohibition on time reads. Define the boundary explicitly, and include reporting and swap work in total frame cost.

Avoid `%f` formatting on PS2's libc in per-frame HUD/debug strings; it has crashed. Use integer fixed-point formatting such as `%d.%02d`.

Chained translucent passes should share one state block. If water splash and engine jets both want lighting off, depth writes off, alpha blend on, `PGL_CLIPPING` off, and edge AA off, the caller should set that state once, call both helpers, and restore once.

Chapter 8 - Depth, Blending, and 2D Primitives

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) — the ZTST pixel test, alpha-blending equation and its $0x80 = 1.0$ factor convention, the texture environment (MODULATE), and the rasterization fill rule all live here.

Two GS facts drive this whole chapter and are worth holding in mind: the GS has no separate "depth test enabled" bit (disabled depth simply is `ZTST=ALWAYS`), and its alpha-blend factor treats 128, not 255, as 1.0. Almost every 2D and overlay bug below is one of those two facts leaking through an OpenGL-shaped API that pretends otherwise.

`glDisable(GL_DEPTH_TEST)` is unreliable on ps2gl for overlay passes. ps2gl's cached flag can say depth is already disabled while the GS draw environment still has a real Z test. For depth-independent overlays, force the pass function instead: enable depth test, set `glDepthFunc(GL_ALWAYS)`, disable depth writes, draw, then restore `GL_LEQUAL`. On the GS, "disabled" depth is effectively an always-pass mode anyway.

Coplanar overlays need a different explicit contract. Put every layer that is physically one surface at the same coordinate, draw the base first, then draw the overlay with `GL_LEQUAL` and depth writes disabled. `LEQUAL` is intentional: equal depth means "this is the authored overlay," while earlier opaque scene geometry still occludes it. Use strict `LESS` only when an equal-depth fragment is supposed to preserve the earlier surface. Do not manufacture painter order with Y nudges; at a wall or mask boundary those offsets become visible silhouettes.

Coplanar does not mean equal raster depth

The tie rule above requires equal **rasterized** depth. Replaying identical transformed triangles satisfies a stronger contract than drawing a shorter sticker on a full-height facade. Different vertices, clipping intersections and diagonals can produce different GS depth planes after quantization, even when their original world plane and matrices agree. `LEQUAL` accepts a tie; it does not repair a fragment that rounded behind its host.

The GS stores XY at 1/16-pixel precision and XYZF2 Z as an integer. The common VU `FTOI4.xyz` instruction does not add four fractional bits to stored Z: GIF packed XYZF2 extraction discards those low four Z bits. Use the live raster scale and depth format when calculating a tolerance. A 224-row field and a 448-row progressive buffer do not have the same pixel-space slope.

This distinction explained persistent entrance/facade fighting in the independent-decal case study. Matching the raw X2 transforms helped but did not eliminate it. Disabling the additive material did not fix it. Reducing tiled U magnitude addressed texture precision but left the depth artifact. The 0.5% reciprocal-depth pull that worked on another platform was below one Z16 unit beyond about 16.4 eye-space units with projection near 0.05. Earlier slope bins also gave near-crossing quads zero bias and capped the result at 128 units.

The hardware-confirmed repair uses a small decal adapter:

1. Transform with the same combined GS-scaled view/projection as the host and clip homogeneous coordinates before dividing, using the renderer's actual near plane and side guards.

2. Derive depth slope from the original face, so a source vertex crossing $W=0$ cannot disable the correction. Exactly eye-coplanar zero-area faces skip individually; they must not disable every other decal in the batch.
3. Add an allowance in **actual GS depth units**, alongside the intended material tier. The measured case-study implementation used $2 + (\text{abs}(dZ/dX) + \text{abs}(dZ/dY))/8 + M \cdot 2^{-22}$, plus a transform-cancellation allowance based on absolute matrix products and reciprocal W ; M is the active maximum depth. Check representable headroom instead of blindly capping a required correction.
4. Submit the chosen NDC depth with identity matrices and explicit $S=u/W$, $T=v/W$, $Q=1/W$. The existing unlit X2 path accepts three texture components, so perspective mapping survives without a microcode change. Carry each material's source fog/alpha through clipping too.

Keep the opaque surface as depth owner when the decal's numerical allowance must not occlude later objects. In the case study, both decal materials tested against opaque depth with writes disabled. Changing writes alone could not fix the initial host comparison; comparison tolerance and depth ownership were separate parts of the repair. Other pass orders need their own analysis.

The reported fighting was corrected on hardware. A supporting model reproduced 2,145 failures for the old 0.5% route and none in 221,849 corrected samples, but did not emulate all VU/GS arithmetic or pixel rules. The numeric allowance is a practical decal tolerance, not proof for every grazing view or nearly coincident unrelated occluder. Staging cost and occlusion coverage require separate checks.

References: GS User's Manual p60 (STQ) and p139 (XYZF2), EE User's Manual p154 (GIF packed extraction), for the packed-coordinate and interpolation contracts.

Alpha and 2D primitives

Untextured alpha-blended primitives do not fade correctly. ps2gl scales untextured vertex alpha by 255, while the GS blend factor treats 128 as 1.0. Upper-half alpha values clamp or over-blend. If something must fade, make it textured with a 1x1 white texture under `GL_MODULATE`.

Stock raylib 2D shapes can select unsupported untextured varying-color state or pay excessive setup. The fork's explicitly admitted colored HUD renderer is a different route. Use a PS2 shape helper that binds a GS-alpha-scaled 1x1 white texture, uses a single uniform color, disables culling, forces correct alpha blend, and flushes so it does not merge into a neighboring varying-color batch.

Do not use raylib's built-in shapes texture for alpha-blended PS2 fills unless it has been GS-alpha-corrected. A 255-alpha 1x1 white texture saturates the GS alpha scale and turns dim quads fully opaque. Also force 1x1 textures to nearest filtering; hardware linear filtering can sample adjacent VRAM outside a 1x1 texture even when PCSX2 appears fine.

The GS quad diagonal matters. Textured quads split along one diagonal showed a 1-pixel seam on glyphs. The working vertex order for raylib's `DrawTexturePro` is `BL, TL, TR, BR`, which makes ps2gl's quad shader produce the cleaner diagonal. Integer snapping, half-pixel offsets, explicit triangles, and the old OpenGL 0.375 translation did not fix it.

Chapter 9 - Geometry, Clipping, Projection, and Display Lists

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) and Ch 4 (Image Data Transmission) for what reaches the rasterizer; VU User's Manual and VU Instruction Manual for the VU1 transform/clip path ps2gl uses; EE Overview Manual Ch 3 (Functional Overview) for the EE→GS data flow the software clipper participates in.

The PS2 has no hardware near-plane clipper the way a desktop GPU does — geometry that crosses the view plane or runs off the fixed-point coordinate range does not get safely discarded, it wraps into garbage. That single absence is why this chapter exists: every working 3D path here is really an answer to "who clips this, and against what guard band," whether that is the EE software clipper, a proven bounding-sphere bypass, or a whole-triangle cull.

Projection range and clipping range must be considered together. Increasing projection near from 0.05 to 1.0 can recover about 20 times the far-depth resolution, but it is free only if every producer sharing that depth buffer already clips at 1.0. For example, if a close effect clips at 0.05, raising the whole scene's near plane to 1.0 removes valid geometry. Giving only distant scenery a different depth mapping also breaks its comparison with unchanged actors and backgrounds. Halving a very distant far plane does not double precision: with near fixed and far much larger, the reciprocal-depth coefficient changes only slightly. Neither adjustment guarantees agreement between differently quantized coplanar triangles; Chapter 8 addresses that separate problem.

Raw world-space `glBegin` inside `BeginMode3D` does not work reliably on raylib4ps2. raylib's camera matrix and ps2gl's current `GL_MODELVIEW` can be desynchronized. Use one of the proven routes:

- display-list models via `rlgl` transforms and `glCallList`;
- software-clipped meshes through `DrawMeshSH` or `DrawMeshSHNear`;
- screen-space/2D effects after `EndMode3D`;
- special eye-space raw GL for perspective textured quads with identity modelview.

For perspective-correct rotating textured quads, use the `DrawMeshSH` recipe: `BeginMode3D` to set projection, flush `rlgl`, set modelview identity, submit eye-space vertices with raw `glBegin(GL_TRIANGLES)`, and let projection produce the `W` used for STQ interpolation. 2D screen-space quads interpolate texture coordinates affinely and wrinkle when tilted.

`SHBasis` must track camera roll, not just yaw and pitch. Build it from camera vectors:

```
forward = normalize(target - position);
right = normalize(cross(camera.up, forward));
up = cross(forward, right);
```

This keeps software-clipped geometry aligned with raylib's camera and `GetWorldToScreen`, especially during banked flight.

`PGL_CLIPPING` has two regimes:

- raw display-list models need it on by default, with a bounding-sphere bypass only when the whole model is provably in front of the near plane and inside a tight guard band;
- software-clipper meshes have already been clipped on the EE, so `PGL_CLIPPING` should be off or `ps2gl` can still reject distant valid draws.

Whole-triangle rejection against near and side guards is an optional effect policy, not clipping: it may drop a visible edge. Use it only where that loss is acceptable. An ordered colored-array path and its fallback must preserve the same bucket order and avoid submitting work twice. Every producer needs its own clipping and capacity contract.

Projective effects must be bounded. Anything using `1/zc` can explode as `zc` approaches zero. PC and DC viewport clipping can hide that; PS2 GS coordinates wrap into garbage. Clamp projected offsets, fade near singularities, and reject NaN/Inf by bit inspection because `-ffast-math` can delete ordinary non-finite guards.

Display lists capture texture handles at compile time. They do not keep the source `Model` alive, but `UnloadModel` frees the textures the list references. Either keep model textures live for the list lifetime or delete/recompile the display list when loading/unloading the model. For a scene-local model, delete its compiled list before unloading the model and recompile after loading it again.

Client arrays queued by `packet.Ref()` outlive the draw function. A frame's double-buffered scratch cursor belongs to the frame owner, never to a material helper. In one failure, resetting it inside an effect draw overwrote an earlier queued skybox and produced broken sky textures. Saving and restoring the cursor would not restore the overwritten bytes. Append behind prior draws and recycle only after the frame owner has proved the referencing work complete, then selected the reusable buffer. Local stack input is safe only when the receiving helper copies it into retained storage before returning. A transactional adapter can stage into unpublished scratch, reserve all of its material passes together and publish only after success; fallback cannot expose a partial stream or reclaim earlier queued data.

Display-list model culling is asset-specific. Record the source winding and set the required state explicitly. Separate invariant installed geometry from per-frame tint or lighting inputs.

Stream per-frame generated geometry through `begin/add/end` helpers. Do not pack several kilobytes of temporary primitive data on the EE stack before calling the backend; the stack can corrupt and surface as an instruction-fetch TLB exception. For a procedural strip or fan, `begin` sets state and selects reusable storage, `add` appends primitives, and `end` clips and draws the accumulated geometry.

Stock lit per-vertex color paths need the established flat-light setup: constant normals, matching directional light, zero ambient/specular and `glColorMaterial(GL_FRONT_AND_BACK, GL_DIFFUSE)`. The fork additionally has an explicit `PGL_UNLIT_TEX_TRIANGLES` route for qualified prepared arrays. Verify renderer selection and optional attributes rather than generalizing either setup to arbitrary immediate-mode state. Clipping must interpolate the color/fog attributes owned by its consumer.

Chapter 10 - Texture Formats, Alpha, and Filtering

Relevant manuals: GS User's Manual Ch 2 (Local Memory) for the PSMCT32/16 and PSMT8/4 pixel-storage modes, CLUT storage, and page/block geometry; Ch 3 (Drawing Function) for texture mapping, the TEXA alpha expansion, and filtering; Ch 8 (Details of GS Local Memory) for addressing;

`ps2textures.htm` as a quicker companion to the PSM/CLUT tables; GS User's Manual Supplement for errata.

GS texture work is a VRAM-residency problem first and an image-quality problem second. The GS has 4 MB of local memory carved into fixed-size pages, and a texture is only cheap if it fits and stays resident; the choice of pixel-storage mode (32-bit vs 16-bit vs 8-bit paletted) is mostly a choice about how many pages you spend and therefore what stays resident. This chapter is that economy, plus the alpha and alignment traps that bite once you leave plain RGBA32.

Size textures to on-screen footprint and GS residency, not source resolution. The expensive case is not "large texture" by itself; it is a texture too large to stay resident, causing eviction and re-upload on the draw that needs it. Oversized HUD atlases both cost upload time and evict other textures.

Use console-scale art as the default PS2 source, not HD sources. HD PNGs can decode to many megabytes in EE RAM and fault when an image decoder walks past allocation failure. Keep the original aspect unless intentionally changing art.

PSMCT16 halves VRAM for opaque art. A 512x512 RGBA32 texture is 128 GS pages and needs a 128-page slot; 16-bit 5551 needs 64 pages. In a map whose largest compatible slot is 32 pages, neither fits without a different asset or layout. Pack in GS bit order: red in the low five bits, then green, blue, alpha in bit 15. Remember the cost: only one alpha bit. Cutout sprites, fonts, and smooth alpha art should not use this path.

PSMT8 paletted textures are the practical workhorse. They use 8-bit indices plus a 256-entry 32-bit CLUT, giving quarter-size texture bodies with full CLUT alpha. Choose per asset: RGBA32 can preserve gradients and smooth alpha without a palette, while 16-bit storage can suit opaque material families.

PSMT8 rules:

- reorder CLUT entries for GS CSM1 layout at build time;
- pack opaque CLUT alpha as `0x80`, not `0xFF`;
- make each texture own its CLUT in the ps2gl fork, because a single global CLUT causes two paletted textures to sample the last uploaded palette;
- reserve kPsm32 GS slots for CLUTs, because `CMMCLut` allocates a 16x16 kPsm32 area and cannot see a slot declared as PSMT8;
- all mip levels of one PSMT8 texture share the base texture's one CLUT.

Every DMA-fed embedded blob needs 16-byte alignment. When multiple texture blobs were packed into one generated `.s`, blobs after a 4-byte size field landed at address 4 mod 16. The EE DMAC truncated the source address to a qword boundary and the CLUT upload read four bytes early, shifting the palette by one entry and turning index zero black. Emit `.balign 16` before every blob, not just once per file.

Smooth color ramps are poor palette-alignment tests. Use a sharp grid probe with distinct colors at neighboring entries. If PCSX2 hardware and software agree on wrong colors, suspect data/alignment, not sampling.

Texture alpha stored in GS memory needs the GS scale. Procedural alpha textures should convert source alpha from 0..255 to 0..128 before upload. Runtime `glColor4ub` is already normalized by ps2gl; do not rescale runtime colors again unless bypassing ps2gl.

Procedural atlas details must also survive the actual texel grid. Even-sized cells have no texel centered at their geometric origin. With a normalized radius spanning -1..1, the nearest texel-center radius in a 16x16 cell is 0.0884; an analytic dot smaller than that can exist in the formula yet produce no sample on PS2. Validate generated radial features at the texel centers of the smallest shipping cell. For light sprites, keep raster-space exposure coverage broad enough to be sampled and tune RGB whitening independently from alpha/energy. If one atlas cell and one quad already express the effect, repair the texture generator rather than adding runtime geometry, passes or adjacent- cell crossfades.

State every asset's filter and wrap mode explicitly. Point-sampled font atlases and 1x1 solid textures should use nearest filtering where exact texel sampling is required. Bilinear 1x1 textures can flicker on hardware. When using raw `glTexParameteri`, pass ps2gl's `GL_REPEAT` and `GL_CLAMP` symbols, not desktop numeric constants; ps2gl's enum values differ.

Skybox and floor UV magnitudes matter on real GS hardware. Large absolute UVs cause wobble because the GS perspective-correct interpolation loses precision as S/T magnitude grows. Keep UVs local to slices or wrap the integer tile count out of scrolling UVs. PCSX2 can hide this.

Anamorphic texture storage is a valid VRAM lever: store one axis squeezed and let GS sampling stretch it back. The loader must report logical dimensions, not stored dimensions, or any code deriving UVs, sprite rects, or atlas frame sizes will compensate in the wrong direction.

Chapter 11 - Manual Mipmaps, Dither, and GS Texture Precision

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for texture LOD selection, mip filtering (MMIN/MMAG), and dithering (DIMX/DTHE); Ch 7 (Registers) for the TEX0/TEX1 fields (MXL, MMIN, K, L, LCM) and the MIPTBP1/2 mip base pointers.

Mipmaps on this stack are a build-it-yourself feature: the GS hardware does them natively through TEX1 and MIPTBP, but ps2gl ships no path to drive those registers in its upstream baseline; the fork supplies the ownership and upload paths described here. It pairs with dithering because both are about fighting the precision loss of a 16-bit framebuffer — one in texture LOD, one in the final store — and both fail in instructive, hardware-only ways that an emulator renders perfectly.

The fork now supports mipmaps as per-texture state. Each pyramid needs TEX1 filter/LOD configuration, valid MIPTBP addresses and resident ownership for all levels. Legacy paths pin separate allocations; qualified packed formats share an owner with proven block offsets and sampler strides. Other shapes retain the legacy fallback.

MIPTBP addresses need not be contiguous. Re-emit each texture's MIPTBP with its settings, rather than one global load-time packet. Multiple pyramids coexist within the slot budget. Deep teardown releases hidden levels and shared owners exactly once. Reconstruct the complete pyramid after a VRAM-layout wipe: sampled-only levels do not lazily bind themselves. Chapter 21 covers lifetime; Chapter 33 covers packed mipmaps and atlas isolation.

The GS supports trilinear (`MMIN = 5`) in one pass. Rules for other console texture units do not determine GS behavior. Real hardware blends mips with limited precision. PCSX2 can show perfectly smooth trilinear gradients while the real GS shows faint stepped bands. Use a mip-tint probe to verify whether MIPTBP/MXL/MMIN are working and where transitions land.

A 16-bit framebuffer needs GS dithering. The GS has DIMX plus DTHE for this, and ps2stuff's draw environment already has a default matrix. Enable dither through the drawenv object, not a raw register write, because ps2gl re-sends the drawenv and will clobber ad hoc writes. Gate the effective value to 16-bit layouts.

Texture-space dither does not fix magnified framebuffer banding. The sun disc case showed why: bilinear magnification low-passes texel-frequency dither back into a smooth ramp, then the 16-bit framebuffer quantizes it again. Screen-space dither or fewer quantizing additive passes may help, but the baked texture dither experiment was reverted.

Chapter 12 - GS and EE Memory Management

Relevant manuals: EE Core User's Manual for the R5900 caches and memory system (why IOP-DMA'd data needs the uncached mirror); EE User's Manual for the DMAC and scratchpad; GS User's Manual Ch 2 and Ch 8 (Local Memory) for the GS page/slot layout the VRAM allocator manages.

There are two separate memory budgets on a PS2 and they fail differently: EE RAM (32 MB, where decoded assets and the geometry buffers live) overflows into TLB faults, while GS VRAM (4 MB of paged local memory) overflows by silently failing to make a texture resident. This chapter is about measuring both honestly — and the non-obvious facts that textures are double-resident by default, and that for VRAM the number that matters is the largest *single* free slot, not the total free page count.

Audit immediate-buffer units and the actual caller override. One ps2gl integration reduced an immediate capacity of 1281024 *vertices to* 321024, reclaiming roughly 12 MB from approximately 16 MB of immediate arrays. A later 16*1024 setting was an application-specific budget, not a recommended SDK default. Count vertices, not qwords, and include retained lists, custom client arrays and the capacity required by their fallbacks before selecting a size.

Use `mallinfo()` for live EE heap readings. On PS2, `uordblks` and `arena` tend to be close because the heap does not return pages to the system, but it is still useful live data.

Textures are double-resident by default: their pixels live in EE RAM as DMA sources and in GS VRAM after upload. This is useful for lazy re-upload after eviction, but it costs memory. If reclaiming EE memory becomes important, be explicit about how an evicted texture can be rebuilt from embedded data.

Add a real GS VRAM query in the fork. `pglPrintGsMemAllocation()` is useful but stdout-only. The valuable live numbers are total pages, used pages, and largest free slot. The largest free slot matters because a texture must fit in one slot. Page count alone does not answer residency.

Default GS slot maps are as important as texture byte size. A 512x512 texture is 32 pages at 8-bit, 64 pages at 16-bit, and 128 pages at 24/32-bit. If the largest free slot is 64 pages, full-res 24/32-bit cannot fit regardless of total free pages.

Chapter 13 - Procedural Effects Under GS Constraints

Relevant manuals: GS User's Manual Ch 3 (Drawing Function) for perspective- correct texturing, blending and coordinate precision; VU User's Manual for transform and clipping paths.

Effects must obey the same clipping, state and lifetime contracts as opaque geometry. Small screen coverage does not imply small projected coordinates, and a transparent texture does not make an unclipped triangle safe.

Coupled geometry and depth ownership

Procedural backgrounds that meet along a shared edge must agree on polygon endpoints and coordinate frames. For a faceted cylinder, the visible wall lies at the polygon inradius between vertices, not at the nominal vertex radius. Using inconsistent slice geometry can expose seams that resemble depth bugs.

Compose coplanar color layers before publishing depth when independently quantized depth would make those layers fight. One background design draws ground color and its overlays, replays ground depth without color, then draws the enclosing sky with strict depth comparison. A haze layer can be composed as background before opaque objects when the viewpoint and scene contract permit it. This is an example, not a universal pass order: views outside the enclosure or below the ground can need a depth-tested alternative. Define which pass owns depth and which later elements must remain visible.

Projected cards, streaks and beams

Projective effects using $1/z$ become singular near the view plane. Preserve the sign of view-space depth, bound projected offsets and fade an effect before its intended representation becomes discontinuous. Taking $\text{abs}(z)$ can put a vanishing point on the wrong side of the screen. Under `-ffast-math`, use a non-finite check that the compiler cannot optimize away, such as exponent-bit inspection, before writing GS coordinates.

A textured ramp with uniform vertex alpha is useful when a selected ps2gl renderer lacks the required varying-color path. Match this representation's brightness and blend behavior deliberately; it is not automatically equivalent to the original vertex ramp.

Long beams and near-crossing cards need a real clipper. A fast path proven for compact distant billboards does not qualify a long thin triangle or an expanded clipped fan. Account for worst-case output capacity and preserve every required attribute. Fog must affect the complete textured result: multiplying vertex RGB alone cannot lift black texels toward a nonblack fog color.

View-plane billboards share the camera's orientation and can reuse one basis per frame. Position-facing billboards instead rotate toward the camera position; they produce a different appearance. Choose the intended geometry explicitly. Either representation must establish its own cull, blend, depth-test and depth-write state. A visible debug triangle through the same route helps distinguish an empty producer from a state path that rejects every fragment.

Avoid unsupported empty immediate batches, but do not equate them with every zero-count GIF packet. Validated VU paths can use `NLOOP=0, EOP=1` completion tags; the packet protocol determines whether an empty packet is valid.

Move invariant work out of the frame

The EE benefits when effect preparation separates invariant geometry from camera-dependent work. For a beam clipped against static scenery, precompute a conservative candidate set, then test only those candidates against the live beam. Prune against the maximum supported sweep or width, not merely its default. Do not bake a cut distance that assumes a camera-facing direction which will change at runtime.

Producer and runtime must share coordinate decoding and all constants used in conservative bounds. Store cached geometry in an explicit stable frame and apply a live origin delta once; paired hidden compensations in both stored vertices and camera transforms are prone to double application.

For animated sprite fields, cache stable positions, phase and amplitude. Reject motion-expanded bounds before evaluating individual animation, then test the surviving animated positions precisely. For sprites inside a faceted cylinder, an inradius-based bound such as `radius*cos(pi/slices)-clearance` accounts for the actual wall. Reuse a renderer only if it already provides the needed blend, depth, atlas and clipping contract. Measure both EE preparation and submission costs; fewer trigonometric calls alone do not prove frame savings.

Separate simulation evidence from rendering evidence

Before debugging an apparently missing effect, prove the simulation emitted it at that timestamp. A borrowed fixed-step clock must restore every affected time value, not only absolute time. A timing error hidden by a capped frame rate can produce a different effect population on an uncapped host and look like a PS2 clipping failure.

Validate frame deltas before updating persistent state. In builds where normal floating-point comparisons retain IEEE behavior, `!(dt > 0.0f) || dt > max` rejects NaN, nonpositive and excessively large values. Under fast-math, retain an explicit bit-level non-finite check. NaN state can otherwise contaminate cameras and projected coordinates across subsequent frames.

Chapter 14 - Input, Debug UI, and Save Data

Relevant manuals: EE Overview Manual Ch 2 for system context (the IOP owns pad and memory-card services, the EE reaches them over SIF RPC). libpad and libmc behavior is ps2sdk-specific; the rules here are practical, from the case study.

Pad and memory-card services cross the EE/IOP boundary. Keep DMA/RPC storage aligned and alive, and initialize services in the owned boot sequence.

Translate libpad into an application input abstraction with an explicit button mapping. Retain access to platform-specific controls and raw state for diagnostics; no particular console-to-console button naming is required.

Diagnostic controls should expose the hardware behavior being investigated: video timing, framebuffer size, aspect, dithering, edge AA, interlace filtering, screen position or fit. A frame-rate target only makes sense together with the application's presentation policy. Do not confuse render rate, field rate and the rate of unique frames reaching scanout.

GS edge AA on opaque models can read as a dark outline rather than soft anti-aliasing. Compare it on hardware with the intended blend and coverage behavior before choosing a default.

For memory-card saves, initialize libmc before GL/raylib init. `mcInit()` calls `sceSifInitRpc(0)` unconditionally; running it late can desync ps2link fileio. Load `rom0:SIO2MAN`, `rom0:MCMAN`, and `rom0:MCSERV`, then `mcInit(MC_TYPE_MC)`. Use the plain modules, not the `X*` variants.

Keep libmc result and transfer buffers alive through RPC completion and 64-byte aligned; static storage is a simple way to satisfy the lifetime rule. The RPC DMA writes into them; stack buffers can corrupt return addresses and surface as instruction-fetch exceptions.

Decode `mcGetInfo` and `mcSync` results carefully. `0` and `-1` can both mean an OK card state for this flow. `-2` is unformatted, and values below about `-10` indicate no card. `mcClose` commits through MCMAN's cache; successful write plus close persisted across power-off in hardware tests.

A historical save investigation found a blocking stdout write changed whether a later ps2link fileio call failed. This is timing-sensitive diagnostic evidence, not proof of a synchronization barrier. A workaround of that kind does not explain the shared-service ordering; fixes must establish readiness and completion rather than rely on print latency. Standalone persistence and dev-loader reliability require separate checks.

A BIOS-browsable save needs a directory with `icon.sys` and an `.icn` model. Missing either appears as "Corrupted Data." The icon model is a tiny animated 3D format with 16-bit fixed-point positions/normals/UVs and a 128x128 BGR555 texture. Write it once and avoid rewriting it on every save.

For `.icn` geometry, the BIOS/mymcplus reference negates Y/Z and rotates about (0, 2.5, 0). An exporter targeting that convention must account for the model origin, winding and texture orientation. Verify the result in the actual BIOS browser rather than assuming a desktop model viewer uses the same coordinates.

Chapter 15 - SPU2 SFX Fundamentals

Relevant manuals: SPU2 Overview Manual Ch 1 (Overview of SPU2) for the ADPCM waveform format and 2-core/24-voice layout; Ch 2 (Sound Generation) for voice generation, the PITCH math (§2.1.2), ADSR envelopes, and mixing/VMIX; Ch 3 (Register List) for the VOLL/VOLR/PITCH/ADSR/KON register fields.

The SPU2 is a local-memory ADPCM synthesizer, not a stream player — it does one thing extremely well (per-voice pitch and volume on samples resident in its 2 MB of RAM) and nothing else. This chapter is how to feed it: the IOP-address gotcha in the upload path, the encoder quality gap that makes a naive ADPCM tool sound like AM radio, and the voice-pool and block-flag conventions that separate a looping layer from a one-shot. Streaming long music is hard enough to get its own chapter (Chapter 16).

Use `libsd/ps2snd` direct voice control for game audio that needs continuous runtime pitch and per-voice register control. `audsrv` is not the right abstraction for that job.

Load both `libsd.irx` and `ps2snd.irx`, in that order. `rom0:LIBSD` is not the EE-facing RPC server, and `libsd.irx` alone is not enough. `sceSdInit` binds to the PS2SND RPC registered by `ps2snd.irx`; without it, the bind loop hangs.

`sceSdVoiceTrans` takes an IOP address as source. EE pointers are meaningless to the IOP module. Allocate an IOP buffer, DMA EE data to IOP, then ask `libsd` to DMA IOP data to SPU2. The SPU destination argument is the SPU byte address cast as a pointer value, not the address of a local variable containing that byte address.

Do not chunk through one reused IOP bounce buffer unless the previous IOP to SPU DMA has fully completed. Reusing it early overwrites in-flight audio data and sounds like gritty radio noise. The simple correct path for resident SFX is one IOP buffer sized for the whole sample, one EE-to-IOP DMA, one `sceSdVoiceTrans`, wait with `sceSdVoiceTransStatus`, then free the IOP buffer.

SPU2 pitch is native and should be used. For 48 kHz samples, `PITCH = 0x1000` means unity playback. The usable range is `1..0x3FFF`, roughly -12 to +2 octaves. Other encoding rates need a corresponding base pitch.

For continuous loops whose amplitude is driven by game mix code, use instant attack and flat full sustain ADSR so `VOLL` / `VOLR` are the actual level. Update pitch and volume per frame as needed.

SPU2 voices are PS-ADPCM. The stock `ps2adpcm` encoder sounded poor because it lacked noise shaping and robust predictor handling. A good encoder should choose best filter/shift per block, noise-shape quantization error, keep predictor history continuous across blocks, and clamp input peaks to avoid overflow.

Loop and one-shot ADPCM block flags differ:

- loops set first block `0x06` and last block `0x03`;
- one-shots set only the final block `0x01` END flag.

One-shots need an explicit allocation and voice-stealing policy; a round-robin pool is one simple choice. Re-keying a still-playing voice steals that playback. Reserve persistent loop and streaming voices

separately, and treat voice numbers as application allocations rather than SDK conventions.

Keep shared mix math shared. The case study moved target volumes, smoothing, ducking, pan, and SFX suppression into common code so PC/web, Dreamcast, and PS2 differ only at the leaf "set this voice/stream" layer.

Live control needs playback identity

A reusable SPU2 voice number is not the identity of one playback instance. If a one-shot is stolen and its slot reused, a stale live-mix record must not change the new sample's volume or stop it. Pair the index with a generation counter, advance ownership when assigning the slot, and validate range, generation and active state before every retained update or stop. Keep reserved loop ownership separate from the recyclable one-shot pool.

This is especially relevant to a layer debugger: disabling new emissions does not silence already-playing voices, while stopping by sample identity can affect overlapping instances. Fade and stop the exact owned playback. Test stolen slots, overlapping instances and stale handles explicitly; source ownership checks and successful ordinary playback establish different parts of the contract.

Chapter 16 - Music Streaming and Custom IOP Modules

Relevant manuals: SPU2 Overview Manual Ch 2 (Sound Generation) for the stream input model, the loop/repeat block flags, and VMIX mixing; EE Overview Manual Ch 2/3 for the EE↔IOP split and SIF RPC; EE User's Manual for the DMAC paths the transfers ride on.

Music is where the SPU2's "not a stream player" nature collides with reality: a song is too big to fit in SPU2 RAM, so you must continuously refill the half of a ring buffer the voice is not currently reading. The hard-won conclusion of this chapter is architectural: keep blocking refill work away from the EE render thread. A custom IOP module is one way to provide that separation; its command, file-I/O and DMA responsibilities still need explicit ownership.

Evaluate the file-I/O and scheduling dependencies of a streaming API before using it. In the case study, ps2snd's `sndStreamOpen` deadlocked the IOP on both `host:` and legacy `mass:`. Its model opens files on the IOP inside blocking RPC/stream machinery, which collides with ps2link fileio and lazy USB mounting.

A custom SPU2 ring streamer can separate these responsibilities:

- two voices, one per stereo channel;
- each voice uses a two-half ring in SPU2 RAM;
- the file is chunk-interleaved `[L chunk][R chunk]...`;
- block flags make the ring self-loop;
- `NAX` tells which half the hardware is currently reading;
- refill the half the play cursor just left;
- rewind at EOF for looping tracks;
- keep VMIX bits routed into the dry mix.

Keep file I/O and SPU DMA out of the render thread. In the case study, an EE-side streamer worked but caused frame hitches and GS margin artifacts during screen changes because file reads and SPU DMAs touched the render thread. Moving file I/O and SPU DMA to a custom IRX made the EE send only play/stop/volume/pause RPCs and eliminated per-frame audio refill work from the EE frame loop.

The IOP module's RPC handler should not do blocking file reads or SPU DMA. It should copy the request, signal a stream thread, and return after consuming the command. The stream thread opens, primes, polls `NAX`, reads, and transfers. Poll with a short `DelayThread`; avoid SPU IRQ file I/O.

Music and SFX share core-wide VMIX registers. Both owners must OR their bits in without clearing the other side. Define the reserved voice masks and coordinate register ownership; a read-modify-write only preserves another owner's bits when competing writes cannot race it.

Custom IRX build traps:

- build with the IOP toolchain and run through `iopfixup`;
- provide your own `irx_imports.h`;
- import `memcpy` / `memset` from `syslib` when using `-nostdlib -fno-builtin`;
- read files on the IOP through `iomanX_*`, not EE `fileXio`;

- avoid `*/` inside block-comment prose in import headers.

For USB, use the BDM/iomanX stack and the `mass0: device: iomanX.irx`, `usbd.irx`, `usbmass_bd.irx`, `bdm.irx`, and `bdmfs_fatfs.irx`, in order. This is not legacy `usbhdfs mass: .` The BDM stack mounts eagerly, but a failed open during startup can mean mounting is incomplete. Retry within a bounded readiness policy while distinguishing missing files and permanent I/O failures from mount progress.

If the EE reads from iomanX instead, add `filexio.irx`, call `filexioInit()`, and use `filexio*`. Define `NEWLIB_PORT_AWARE` before including `filexio.h` if needed and do not mix those descriptors with `newlib open()`.

When reading memory the IOP DMA'd into EE RAM, use the uncached mirror `addr | 0x20000000`. The EE data cache may still hold old lines.

Plain burned data CD/DVD media are illegal media on stock or FMCB-only consoles.

`sceCdGetDiskType()` reports illegal media and reads return zeros. `cdrom:` via `cdfs` is useful for authenticated discs, modchip scenarios, or emulator ISOs, not for unmodded network-boot dev. Use USB or HDD for dev removable storage.

Chapter 17 - Validation Doctrine

Relevant manuals: all of them, but this chapter is mostly empirical — it is about how to *use* the manuals and the emulator together rather than any single register.

If the handbook has one meta-lesson, it is this chapter: a PS2 emulator and a PS2 console are different witnesses, and the gap between them is exactly where the expensive bugs live. The discipline below — validate the right things on hardware, prove an object exists before debugging why it "disappeared," compare PCSX2's hardware and software renderers to localize data-vs-sampling bugs — is what turns a multi-day ghost hunt into a one-screenshot diagnosis.

Validate PS2 video modes on hardware. PCSX2 can hide bad `magH`, color/Z page mismatches, overscan position, and display-register mistakes.

Validate texture precision and filtering on hardware. PCSX2 can hide skybox UV wobble, real GS trilinear quantization, and 1x1 linear-filter VRAM fetch artifacts.

Validate standalone boot separately from ps2link boot. ps2link can mask missing IOP reset/SBV setup and can introduce its own memory-card/fileio interop hazard.

When a renderer "loses" an object, first prove the object exists at that instant. For example, a recorded-sequence investigation spent time on clipping and draw state before revealing a simulation clock error: the supposedly missing projectiles were no longer being submitted.

When diagnosing GS margin flashes, distinguish framebuffer contents from `BGCOLOR`. A flash confined to the margins directs investigation toward display state; clearing the active framebuffer cannot change those pixels.

When diagnosing color/palette corruption, compare PCSX2 hardware and software renderers. If both match the same wrong output, suspect uploaded data, DMA alignment, or CLUT binding.

When optimizing, bisect by pass before inventing abstractions. Text looked like the HUD hog; rect pipeline reconfiguration was the larger cost. The GS and ps2gl are often state-bound, not vertex-bound.

Do not publish speculative lessons as proven facts. Keep a working engineering log if you like, but promote entries into a public manual only after they have survived real validation.

Chapter 18 - Extending ps2gl and ps2stuff at Ownership Boundaries

Relevant manuals: GS User's Manual Ch 7 (Registers) for the register fields the fork drives — `PRIM.AA1`, `DTHE / DIMX`, `TEX1`, `MIPTBP1/2`, `DISPLAY` — and Ch 5 (CRTC) for the display state the runtime-mode helpers touch; EE User's Manual for the DMA paths; VU User's Manual for the VU1 context behind ps2gl's clipping and immediate rendering.

A PS2 graphics-library extension must live where its state and lifetime are owned. Memory queries belong beside the allocator; persistent GS state belongs in the packet that restores it; upload helpers must define ownership of retained EE data as well as GS storage.

The APIs below are examples from a ps2gl/ps2stuff fork, not standard ps2gl interfaces or an installation recipe. Their useful feature is the dependency boundary. A different fork can implement the same contract under different names.

ps2stuff owns the GS memory allocator, so memory reporting starts there. `CMemManager::GetMemInfo()` walks locked slots and every slot list, accumulates total carved pages, counts bound or locked slots as used, and tracks the largest single free slot. The largest free slot is the number to care about. A texture allocation must fit in one compatible slot; total free pages can lie to you.

ps2gl exposes that as a C-facing wrapper:

```
extern void pglGetGsMemInfo(int *total, int *used, int *largestFreeSlot);
```

The dependency chain is:

```
application
-> ps2gl: pglGetGsMemInfo()
-> ps2stuff: CMemManager::GetMemInfo()
-> ps2stuff: CMemSlotList::AccumMemInfo()
```

For display positioning, the ps2stuff fork adds a narrow DISPLAY-only register push. The point is to shift the raster without re-sending unrelated display state such as `PMODE`, `DISPFB`, or `BGCOLOR`. The example helper writes `DISPLAY2`, matching ps2gl's active read-circuit assumption. A general API should choose `DISPLAY1` or `DISPLAY2` explicitly.

The corresponding ps2gl API is:

```
extern void pglSetDisplayOffset(int screen_x, int screen_y);
```

The dependency chain is:

```

application
-> ps2gl: pglSetDisplayOffset()
-> ps2gl: CDisplayContext::SetDisplayOffset()
-> ps2stuff: CDisplayEnv::SetDisplay2()
-> ps2stuff: CDisplayEnv::SendDisplayPos()
-> GS DISPLAY2

```

Runtime video-mode control is exposed through:

```
extern void pglSetVideoMode(int interlaced, int overscan_mode, int screen_x, int screen_y);
```

This updates ps2gl's display interlace state, reprograms FB2, and uses different display magnification for interlaced and progressive output. In the progressive path the fork uses full buffer height and `magH = 2`, fixing the hardware bug where `magH = 4` made DTV output exactly twice too wide while PCSX2 hid the problem. This API does not rebuild GS memory layouts. Applications that change dimensions or pixel formats still need to rebuild draw/display buffers and slot maps themselves.

Centered viewport scale exists as:

```
extern void pglSetViewportScale(float sx, float sy);
```

It persists `VpScaleX/Y`, folds them into the draw context's GS scale, and updates the drawenv scissor to a centered sub-rectangle. This is not a general OpenGL viewport replacement. It is a PS2-friendly screen-fit/squish primitive for overscan and CRT calibration, bounded to $0 < s \leq 1$ in the case-study fork.

GS edge anti-aliasing is exposed as a ps2gl capability:

```

#define PGL_EDGE_AA 3
pglEnable(PGL_EDGE_AA);
pglDisable(PGL_EDGE_AA);

```

The fork tracks the draw-context state and sets `GS::tPrim.aa1` when giftags are generated. This exposes the GS primitive edge AA bit. It does not solve texture shimmer, texture minification, or aliasing inside textured surfaces.

GS dithering is exposed through the drawenv object:

```
extern "C" void pgl_enable_dither(int enable);
```

The important bit is not the function name; it is the route. Dither must live in the `CDrawEnv`, followed by `DrawEnvChanged()`, so ps2gl re-sends the correct state. A raw register poke is temporary and will be overwritten by the next drawenv send. A consistent public API could use `pglEnableDither()` or a `PGL_DITHER` capability.

Per-texture CLUT ownership is the most direct correctness fix. The original manager-global CLUT model fails with multiple simultaneous PSMT8 textures: the latest `glColorTable()` effectively changes the palette for other indexed textures. The fork gives each `CMMTexture` an owned `CMMCltut`, has `SetCurCltut()` attach the palette to the currently bound texture, and has `UseCurTexture()` load that texture's own CLUT. Display-list capture, palette updates and deletion must preserve that association; ordinary binds alone do not cover those lifetimes.

The PSMT8 upload convenience API is:

```
extern "C" unsigned int pgl_create_index8(  
    const void *indices,  
    int w,  
    int h,  
    const void *clut  
);
```

It wraps the correct `GL_COLOR_INDEX` / `GL_UNSIGNED_BYTE` upload plus a 256-entry RGBA CLUT upload. The assumptions must be documented: the CLUT is 16-byte aligned, is already in the storage order expected by this path, and carries GS alpha conventions.

Manual mipmaps are the largest architectural change. PSMCT16 uses:

```
extern "C" unsigned int pgl_create_mip16(  
    void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    int kbias,  
    int min_filter  
);
```

PSMT8 uses:

```
extern "C" unsigned int pgl_create_index8_mip(  
    const void **levels,  
    const int *lw,  
    const int *lh,  
    int count,  
    const void *clut,  
    int kbias,  
    int min_filter  
);
```

Mip creation configures base TEX1 and retains MIPTBP in the texture's settings packet through `SetMipLevels` / `SetMiptbp`. Legacy creation pins hidden levels; qualified packed formats share an owner. A one-shot global MIPTBP write cannot represent multiple texture owners.

`pgl_delete_mips(baseId)` releases registry-owned hidden levels/shared packed ownership before the appropriate base release. Preserve each format's teardown contract; packed non-mip textures can use their ordinary unload route. Coexistence is bounded by resident slot capacity, not one live pyramid.

Legacy VU build systems may depend on a macro preprocessor absent from the host toolchain. A replacement must define its supported subset: includes, macros, conditional assembly, repeats, equates and parameter syntax can all affect emitted code. One fork used a Python implementation of the required `gasp` subset. Compare emitted microcode and retain reproducible module inputs; Chapter 22 covers scheduler and instruction hazards.

A linked-library marker identifies the implementation in an executable when installed archives and local forks coexist. It proves provenance; state, ownership and hardware behavior still need validation.

Chapter 19 - Delivering Vertex Fog to the GS

The GS blends a fog color using an interpolated per-vertex factor. A library can expose this without a separate geometry pass when its vertex packets already carry XYZF2. This is an integration contract across packet format, primitive state and VU1 output; the hardware unit alone is not enough.

The fog byte is already in the wire format. A GS vertex in the XYZF2 form carries an 8-bit F field alongside X, Y, Z. Fork VU1 kernels that pack XYZF2 already emit the qword — they just write F as zero with `PRIM.FGE = 0`, so the unit is dormant, not absent. Enabling fog is three edits, not a new pipeline: set `FGE = 1` in the gifttag PRIM (which needs `PRMODECONT.AC = 1` so the PRIM fields are honored), set the global `FOGCOL` register, and write a real F per vertex in the kernel. `F = 255` is minimum fog (a $\sim 1/256$ residue remains — true no-fog is `FGE` off), `F = 0` is solid `FOGCOL`. The GS DDA-interpolates F across the triangle exactly like color.

Watch the ADC packing. F lives in the XYZF2 W-word at bits 11:4 (the ADC/kick bit is 15). Kernels that set ADC with the classic "add `0x7fff` and let the carry land in bit 15" trick fill bits 11:4 with garbage — harmless while `FGE` was off, but with fog on that garbage renders as per-triangle fog bands. Mask the ADC word to bit 15 and OR in your F (an `ftoi4` of the fog fraction is already the `<<4` packing). A red `FOGCOL` with `FGE = 1` is the one-boot probe that proves the whole wire path before you touch the fog math.

Fog params ride the kernel context — mind the qword under it. Generated VU1 microcode addresses its scratch through negative offsets from an XTOP-derived pointer, and the context's last qword can sit flush against the double-buffer base. A one-qword underflow then stomps that slot every frame. This is invisible for two decades when the stomped slot holds something tolerant (an unused clip constant); put your fog parameters there and it zeros them every frame — solid fog, insensitive to every parameter you change. In the affected layout a reserved pad qword separated the context from the buffer machinery. Establish the actual address range and correct the overlap; arbitrary padding is not a general repair. The bisection that finds it: hardcode the fog parameters as microcode immediates. If fog then works, the math is right and the bug is delivery, not computation.

GS fog is screen-space, not perspective-correct. The interpolated F is linear in *screen* space, where a per-pixel fog unit (the Dreamcast PVR, say) fogs from true $1/W$. On a large receding surface with few vertices this diverges badly: a single center-to-rim ground fan viewed from a low camera reads as solid fog, because perspective compresses the far ground into a thin screen band and the screen-linear lerp paints mid-fog onto the near pixels. The fix is geometric — sample F where the depth actually changes. Subdivide the surface so vertices land at real depths: radial rings for a ground disc (denser across the fog band), and segments along any long wall receding in Z. Do **not** switch F to a $1/W$ metric to "fix" the interpolation — it will interpolate perfectly and then no longer match a reference curve built on eye-Z. Sony's own fog macro suite (`FogSetup` / `VertexFogLinear` / `CreateGsFOG`, in the toolchain's VCL includes) is the reference for the kernel side; it is worth reading before reinventing it.

Manual reference: GS User's Manual, Drawing Function (fog, the F field, PRIM/ PRMODECONT); Image Data Transmission (gifttag REGS and XYZF2 packing).

Chapter 20 - Interrupt Completion, Vertex Alpha, and Batched Submission

Three unrelated-looking problems that all trace back to how a 2001-era fork feeds the GS: an intermittent whole-machine freeze, alpha that silently dies, and an EE that loses to a slower CPU. All three are pipeline-shape problems.

The interrupt handler that services one event freezes the machine. A GS interrupt handler written as `if (SIGNAL) ... else if (VSYNC) ...` services exactly one event per interrupt. But the GS holds its interrupt line while *any* unmasked CSR bit is set, and the EE interrupt controller latches on edges: if both events are pending at once (or a second arrives during the handler), the un-serviced bit keeps the line high, no new edge ever latches, GS interrupts stop, and the next wait-on-render-or-vsync semaphore sleeps forever. The EE is wedged; the IOP and its network loader stay alive, so the machine looks half-dead and a hardware reset combo still works — the exact signature. It is a per-frame dice roll that gets worse as frames get longer (a ~17 ms frame drifts the end-of-chain signal across vblank). Fix: drain in a loop — service *and* clear every set CSR bit, re-reading CSR until it is clean, then exit. Diagnosing this needs a deadman watchdog (below), because by the time you notice, the main thread is already asleep.

The color breadcrumb, escalated: a watchdog that names the wedge. Painting a live GS `BGCOLOR` per checkpoint scans out as rainbow margin bands — do not do that on a shipping frame. Instead *latch* a checkpoint color, and let a vblank interrupt handler (which still fires while the main thread is parked on a dead semaphore) stamp the latched color into the raster margins after the frame counter has stalled for a few seconds — and *blink* it against a second color read live from the DMA/VIF/GS status registers (DMA busy plus VU1 executing = VU1 hung; DMA idle with a signal pending = an unserviced interrupt; idle with nothing pending = the signal was lost). The checkpoint color says *where* the EE got to; the machine-state color says *why* the chain never finished. This single tool found both the interrupt-drain freeze above and a wrong-microcode VU1 wedge.

An uninitialized bitfield is a heap-layout lottery crash. A renderer registry that initializes its requirements struct field by field can leave a padding bitfield (`unused : 12`) holding heap garbage. Capability matching never sets those bits, so every match fails, and a not-found path that indexes one past the table — guarded only by an assert that compiles out in release — makes a virtual call through garbage. Whether it crashes depends on that build's allocation luck; it can hide for twenty years. Zero the whole struct before the field init, and give the release build a real no-match guard that keeps the current renderer and prints the requirement bits (those printed bits are what expose the garbage). Note the guard is a tripwire, not a fix: "keep the current renderer" on a genuine mismatch feeds the wrong microcode and can wedge VU1.

Lit renderers discard per-vertex alpha in the last instruction before the GS. A stock lit vertex kernel sets each output vertex's alpha from the *constant* material diffuse, computed once before the loop; the per-vertex color path only ever drives RGB. So anything that animates alpha through the per-vertex-color path — the usual workaround, since there is often no unlit per-vertex-color renderer — renders fully opaque, with the blend state and the alpha bytes all provably correct and zero visual effect. The alpha dies in `finish_colors`. Fix is a kernel variant that loads each vertex's own alpha (the `.w` the stock loader skips), scales it into GS 0..0x80 range, and merges it into the output color. Make it

backward-safe: an opaque stream baking alpha $255 \rightarrow 1.0 \rightarrow 128$ must come out byte-identical, and constant-material renderers keep the stock path.

A compound renderer must declare whether its second material inherits alpha. Shared geometry does not imply shared material semantics. Chapter 26 develops the case where an input alpha lane carries fog instead of opacity. A dual-context/dual-texture kick can reuse one geometry stream for an opaque base and an emissive overlay, but shared vertices do not define the semantic alpha relationship. In a settled draw, vertex alpha may carry base-light data and the emissive layer must ignore it; in a spawn fade, both materials must share the host transition. Pass an explicit `second_material_inherits_vertex_alpha` policy from the call site. Blend mode cannot infer it, and silently reusing one alpha source turns a performance optimization into a material-coupling bug.

Pipeline shape beats clock speed: batch the clipper output. A 294 MHz EE lost to a 200 MHz SH4 on the same scene, because the PS2 path re-emitted every software-clipped vertex through per-vertex immediate calls (each a call chain plus several buffer appends) while the other machine submitted one batched client-array draw. The fix is the same on the EE: append clipped vertices to DMA-visible scratch arrays and flush one array draw per surface. The fork ref-DMA's client arrays straight to VU1 (float-only, tightly packed; lit paths still need a normals array — a single shared constant normal works). Two disciplines: the scratch remains immutable until referencing chains complete; double-buffer selection is safe only under that ownership contract. Adjacent compatible draws may merge without changing authored order; do not globally sort overlapping geometry merely to reduce state changes. The payoff beyond speed is memory — the immediate-vertex buffers shrink from a whole-frame worst case to HUD scale, refunding megabytes.

Manual reference: GS User's Manual, CRTC and the CSR/IMR interrupt registers; Drawing Function (alpha blending, the 0..0x80 alpha convention); VU User's Manual for the kernel scratch model.

Chapter 21 - Coexisting Mipmaps and the Long-Run Reset

Two topics that both surface only after the easy path works: making more than one mipmapped texture legal, and measuring whether repeated teardown conserves memory.

MIPTBP is per-texture state — put it in the settings packet. A first mip implementation that writes `MIPTBP1/2` once, globally, via a raw register packet makes "exactly one mipmapped texture may exist" a load-bearing invariant — fine for a lone floor, fatal the moment a scene wants several mipped tiles. But `MIPTBP` is context state exactly like `TEX0 / TEX1 / TEXA`: its home is the texture's own settings packet, re-sent on every bind. Bake the pyramid addresses into each texture at create time and multiple pyramids coexist within the resident slot budget; textures with no mips carry a zero `MIPTBP` the GS never dereferences. One corollary bites on a VRAM-layout switch that wipes the slot map: a fork re-uploads a texture lazily on its next *bind*, but pyramid levels are never bound (only sampled via `MIPTBP`), so after a wipe the re-uploaded base samples its levels from reassigned memory — correct up close, wrong in the distance. Every mipped texture must be deep-released before such a switch and recreated after.

Locked mip pyramids need slot and page budgets. In the legacy route, each mip level is its own pinned allocation from the fixed slot partition, so N mipped textures cost $N \times (\text{levels} - 1)$ additional pinned mip slots plus base/CLUT demand — independent of the trivial VRAM the texels occupy. A batch of tiles with full pyramids can demand far more one-page slots than the pool holds; the locks then starve the CLUTs, which LRU-fight the leftovers and produce full-screen palette corruption while untextured and CLUT-less draws stay normal (the diagnostic tell). Before enabling mips on a batch of tiles, count $\text{tiles} \times (\text{levels} - 1)$ against the small-slot pool and evaluate owner classes. Qualified packed routes instead share an owner; count their CLUT demand and preserve teardown. Recarving the map requires proof that all other resident demand still fits.

A 1-bit alpha key can be mipmapped — if you pick the filter for the blend. The blanket rule "never mip a 1-bit-alpha texture" exists because a box filter averages the key toward zero. A custom mip builder escapes it, but the correct filter depends on how the tile draws. A punch-through (replace) tile wants RGB averaged over the *covered* texels only and the key re-thresholded at $\geq$ half coverage — it conserves coverage, so a minified field of holes reads as a solid bright surface. An additive (light) tile wants a plain box average (the black gaps pull RGB down) with the key set at *any* coverage — it conserves energy, so a far field of small lights dims smoothly like real distant lights. The whole- pyramid alpha behavior is gated by that texture's `TEXA` (the base texture's bind carries it; levels are only sampled). And a related default to know: a fork's `TEXA` often ships as "identity" (`ta0 = 0x80`), which expands a 16-bit texel with alpha bit 0 to alpha 128 and passes a mid-threshold alpha test — so punch-through tiles render as solid quads until you override `TEXA` per texture (`ta0 = 0`).

The long-run reset tests conservation within its instrumentation. A "reset the whole game" path — unload every asset, re-init, reload — is rarely a shipping feature, but a flat per-cycle delta supports ownership for measured pools. It cannot prove absence of every leak or cover pools absent from the instrumentation. Instrument it: after the post-unload drain each cycle, snapshot system RAM (`mallinfo().uordblks`) and live GS/VRAM (a fork memory query on PS2, equivalent counters on comparison platforms). Judge the delta against a *running minimum* — the true drained floor — not

against the previous cycle: a real leak raises the floor monotonically, while allocator fragmentation oscillates above a stable-or-dropping floor (the bookkeeping can even read below the steady value — not a leak). Comparing to the last cycle cries "leak" on a conserving-but- fragmenting run. When a reset OOMs after N cycles, the instrument names the growing pool and you hunt one leak at a time.

The leak the reset exposes: your fork keeps the texels pointer. The largest such leak found here was ownership, not a missing free. A raylib-style loader allocates a DMA texels buffer per texture and hands it to the fork's `glTexImage2D`; the fork *keeps that pointer* — it re-DMA's the texels from RAM on every GS-LRU re-upload, so the buffer is not a transient staging copy and the intuition "the GS has it now, free the RAM" is wrong. But if the fork's texture never had its "free image on exit" flag set, deleting the texture frees the GS slot and never the RAM — a quarter-megabyte per texture, a megabyte per reset cycle, invisible to the loader's own unload path because the leak lives inside the fork's ownership model. The fix is an explicit ownership transfer (a `take-ownership` call after `glTexImage2D`, backed by a "free on exit" setter on the texture object), applied narrowly so paths that manage their own buffers stay untouched. The general rule: any allocator that hands the fork a buffer and then forgets it is leaking — the fork frees the image only if told it owns it. Two smaller siblings round out the set: model/material textures need a *deep* unload (dedup shared ids) rather than the desktop assumption that unloading a model frees its skins, and stage-independent install helpers that rebuild a texture every call must bake it once (`if (already_built) return;`) or they leak one per reset.

Manual reference: GS User's Manual, Local Memory (page geometry, TBP/MIPTBP addressing) and Drawing Function (TEXA, the alpha test).

Chapter 22 - Writing VU1 Microcode Through VCL: The Clip Renderer

Relevant manuals: VU User's Manual Ch 3 (flags, hazards, pipelines) and the VU Instruction Manual for per-instruction flag/latency tables; GS User's Manual Ch 3 for the packed GIF vertex formats the renderer emits.

Clipping can move from EE preparation into a custom VU1 renderer while retaining the contract from Chapter 9. One ps2gl implementation performs five-plane Sutherland-Hodgman: the near plane plus four guard-band side planes. The EE submits eye-space triangles; VU1 classifies, clips, interpolates STQ and emits GS packets. In a historical city workload, the measured pass changed from 11-20 ms to about 9-14.7 ms at higher vertex loads, with EE draw-call time around 4-5 ms/frame. These are workload observations, not a universal cost bound. The extension seam is useful, but the tested VCL toolchain required careful inspection of its scheduled output.

The extension seam needs no changes to ps2gl's renderer selection: `pglRegisterRenderer` plus `pglRegisterCustomPrimType` (a `GLenum` with bit 31 set), keyed to a user-properties bit with a requirements mask of `~0xffffffffull` so matching ignores all standard state. Three traps: the geometry block fills per-prim metadata only for standard prims (set `NumVertsPerPrim` and friends yourself in `DrawLinearArrays`); the giftag builder cannot map a custom enum (pass the equivalent standard prim down); and a failed match silently keeps the *current* renderer, which renders identically if your microcode started as a clone — prove selection with the renderer NAME, not pixels.

Variable-output geometry (one triangle in, up to six out) breaks the stock 1:1 pipeline assumptions. The pattern that works: store each output triangle optimistically at the output cursor and commit (advance cursor and a vertex counter) only if kept; after the loop, rebuild the giftag from its template with the real count and store it over the packet head before `xgkick`. A buffer whose triangles all drop kicks a bare `NLOOP=0 EOP=1` giftag — legal and hardware-verified. Stock lighting passes walk output 1:1 with input and cannot survive 1:N; resolve color in the main loop.

The following failures were observed in the tested VCL toolchain. Verify the emitted program when using these techniques with another version:

- 1. vcl does not model VU-memory aliasing between pointer registers.** It will hoist one pipeline stage's loads above the previous stage's stores to the same address through a different pointer — the consumer reads the previous triangle's data, and which qwords hoist is a scheduler roll of the dice per regeneration (positions can survive while texture coordinates die). One workaround marked each store-to-load seam with a branch to the next line: the label is a basic-block boundary vcl cannot schedule across, and its late peephole erases the branch. This is a scheduler boundary, not a physical VU hazard fence. Verify final store/load order, branch targets and store-to-XGKICK latency in the generated image; labels and assembler exit status are insufficient.
- 2. vcl "regenerates" MAC flags with ABS, which sets no flags.** Never use `fmand` for a new sign test; when the scheduler separates producer and consumer it inserts a flag-inert ABS and the test silently reads stale flags. Extract signs deterministically: clamp a copy to ± 2047 , `ftoi4`, `mtir`, mask bit 15.

3. Pipeline Q with multiple consumers is schedule-fragile. Two `mulq`s off one `div` are only correct while the scheduler keeps both inside that `div`'s window — the next regeneration may not. Land Q in a register at the divide (`addq.x qreg, vfoo0, q — vfoo.x` is zero) and broadcast-multiply from the register. Note `vcl` rejects an explicit `waitq` outside RAW mode; the single-consumer `addq` is the whole fix.

Two hardware traps that cost hardware round-trips: `vfoo.w` is ONE, so building a `.w` constant additively on `vfoo` silently adds one (the near plane became `1+near`); and `ps2gl`'s buffer splitter can hand a triangle-list microcode `3n-1` vertices (an odd-split adjustment from the strips era), which wedges any vertex loop that terminates on pointer equality — VU1 spins forever and the console hard-hangs. Round EE-side per-buffer caps to a multiple of six AND make the loop overshoot-proof (terminate on "remaining less than one triangle").

Validation doctrine for microcode (extends Chapter 17): a standalone test ELF with a build tag banner; textured validation via a procedural checkerboard (texcoord bugs are invisible untextured); geometry that targets the mechanism (tilted discs straddle the near plane, flat discs pop whole for the `NLOOP=0` case, a static marker separates "draws nothing" from "culled everything"); every per-buffer cap exercised by a draw that spills it; and the new path run with `ps2gl` clipping disabled so its own protection carries the load. Know the reference column's limits: stock `ps2gl` silently DROPS guard-band-busting triangles, so the stock side of an A/B can be blank exactly where the interesting cases are. And two GS-level phantoms that mimic clipper bugs: overlapping coplanar geometry from two pipelines z-fights into a static horizontal comb (the two paths round depth differently), and a single giant triangle with huge UV magnitude streaks on the GS's reduced-precision STQ interpolation no matter who clipped it — test with game-scale tiles.

Manual reference: VU User's Manual Ch 3.3.5 (instruction flag table — ABS sets none) and Ch 3.4 (hazards: Q/ACC/I have no interlocks; waitq stalls both pipes).

The output cap is an input contract

A clipping microcode's output capacity is part of its input contract. The original renderer guarded a 39-vertex cap by dropping an overflowing fan and the remaining input. This prevented overwrite but lost valid geometry: rejected behavior, not a lossless fallback. Typical wall counts did not prove a bound on every clipped batch.

It is structurally unsound for giant polygons, and it fails silently. A screen-covering triangle straddles three or four frustum planes at once, clips to a 5-7 vertex polygon, and fans back out as 9-15 vertices; a few of those sharing one 8-triangle input buffer sail past the cap, and whole fans vanish. The symptom on hardware is huge missing triangles that come and go with camera position — whichever triangles happen to share a buffer decide the frame. In one scene this appeared as vanishing ground where no overlapping geometry concealed missing fans.

Three durable conclusions. First, the cap is part of the renderer's interface: a stream whose primitives can individually fan past the remaining budget must not ride that renderer, however rare the overflow — "rare" is exactly what "missing at times" looks like. Second, EE pre-clipping can bound the VU input for a small exceptional stream: every submitted triangle arrives inside the same clipping volume, eliminating further expansion. Its preparation cost must still be measured. Third, a silent-by-design drop deserves a counter — two guards whose only observable output is a missing wedge cost a hardware

session to localize; a dropped-fan count in the log names the failure in one run. Rerouting identical geometry through independent renderers is a useful diagnostic: holes that disappear under another clipping path localize the failure more strongly than a screenshot of the failing path alone.

Refinement: preserve the input batch and spill output inside VU1

Pre-clipping a large ground stream did not establish that ordinary wall quads fit a 39-vertex output budget. A later frozen-camera A/B found a wall triangle missing in the custom compound renderer and its compact-input variant but present with independent clipping paths. Even small input primitives can expand into an unsafe *batch* when near and side planes intersect them together.

Splitting correctly before DMA repaired the triangle and regressed the frame. Conservative and exact-count splitters fragmented the normal four-descriptor VU activation toward one descriptor: the wide path rendered about 10,029 verts at sync 1.74 ms / city 7.95 ms / 60.0 fps, while a safety splitter rendered fewer verts yet reached sync 4.30-4.58 ms and 56-57 fps. Other variants reached 6.22 ms sync / 48.5 fps. The lesson from Chapter 23 applies inside the clipper too: submission shape can dominate arithmetic and vertex count.

The lossless solution leaves the wide input batch alone and spills variable output between two VU-memory arenas during one microprogram invocation. The x2 buffer half has 292 free qwords. A compound arena needs two GIF tags; each emitted vertex consumes three wall qwords and three window qwords. Two arenas therefore obey:

$$4 + 6 * (A + B) \leq 292$$

A=30 and B=18 use the region exactly. A triangle clipped by five convex planes becomes at most an eight-vertex polygon (3 + 5) and a six-triangle fan, or 18 output vertices, so the smaller arena always fits one whole fan. When the current arena lacks room, VU1 closes and XGKICK s it, continues in the other arena, and alternates after the next kick. Arena reuse must preserve PATH1/XGKICK consumption ordering. Alternation alone is no general completion proof; never overwrite a packet still being consumed. The ordinary eight-triangle/24-vertex batch fits A and keeps its original one-activation, one-compound-kick path.

Do not hold arena address/capacity in integer registers across a five-plane S-H clipper; VCL ran out of VI registers. Reconstruct the current tag only at a spill point from `next_output - 3*out_count - 1`. After regeneration, verify generated store/load and kick order, the exact-image hash, decoder relocation, and combined micro-memory budget. For scale, the tested compound clipper occupied 810 instructions and its compact-input decoder another 56, starting at PC810 and tail-calling PC6: 866 of VU1's 2048 instruction slots. These addresses describe that image, not stable entry points for another renderer.

Hardware validation repaired the frozen wall angle and held every steady-state 10-second window at 60.0 fps through a 190-second idle traversal. The dense windows remained at sync 1.73-1.74 ms with roughly ten thousand vertices per frame. These historical window averages do not prove every-frame completion. Requalify exceptional streams before moving them to another renderer: output capacity alone does not establish parity or a performance benefit.

Chapter 23 - Separating Preparation Cost from Submission Cost

Relevant manuals: GS User's Manual Ch 3 (drawing function, per-primitive `ABE`, `ALPHA_1`) and Ch 4 (image data transmission — the GIF packet formats each kick emits); EE User's Manual (DMAC/VIF/GIF paths) for what a "flush" actually costs.

Moving work to VU1 helps only if it removes the bottleneck. A PS2 draw path contains EE preparation, packet construction, DMA/VIF transport, VU execution and GS work, with overlap and waits between them. Instrument boundaries before choosing the next offload. The following two-material facade example shows how measurement can expose duplicated work.

Two counters, one question

A per-pass timer answers *which* pass is expensive. It cannot answer *why*: a pass that spends its milliseconds building DMA chains, stalling on VIF1, or waiting for VU1 looks exactly like a pass that spends them in EE arithmetic. The distinction matters enormously, because the two have opposite cures.

The fix is a second counter wrapped around the single choke point every batched draw exits through - `glDrawArrays`. Accumulate time inside the call and its draw/vertex counts. This is EE wall time in submission, including construction and waits reached there, not a direct GPU execution timer. Remaining pass time includes preparation, wrappers and state work. Count custom paths as well as generic `DrawLinearArrays`. Completion, queue-ready and presentation records supply separate evidence, not an exactly additive ledger.

In that historical city workload, roughly four milliseconds were measured inside `glDrawArrays` against eight elsewhere in the pass, at about ten thousand vertices. Source inspection attributed much of that residual to EE preparation and memory traffic. That is on the order of **280 EE cycles per vertex** — an order of magnitude more than the arithmetic in those loops can account for. The measured bottleneck in that experiment was attributed primarily to memory traffic: reading vertices that had fallen out of cache, and writing them again into uncached DMA memory.

Two consequences followed immediately, and both were unwelcome.

First, a chunked display-list design — compile static slabs of the city once, call them per frame, let VU1 do everything — was not merely difficult on this fork; it did not remove the measured bottleneck. It removed arithmetic and left the per-vertex read and copy exactly where they were. (It was also structurally impossible: ps2gl's list manager is a bump allocator that wraps at four thousand IDs and asserts on collision, each list lazily claims four 256 KB DMA buffers, frees defer a frame, and colors and texture binds bake at compile time. These properties favor stable, reusable geometry. A streaming or frequently retinted scene needs another lifetime and state strategy.)

Second, a "raw submit" fast path — skip the clipper, copy vertices straight into scratch — was measured and moved nothing, for the same reason. Both experiments were built, measured, and reverted. The counter had predicted both outcomes.

What the counter *did* endorse was the opposite move: touch fewer vertices.

One transform, two kicks

The city drew each building wall twice. The first pass laid down an opaque wall, its texture modulated by a per-building tint. The second pass drew the lit windows over it, additively, from the same vertices with a different texture. Identical geometry, submitted, transformed, and clipped twice.

The Graphics Synthesizer offers no way to avoid the second *rasterization*: it has no multitexturing, and no `TFX` mode tints a wall texel while sparing a window texel beside it — which is precisely why the window pass exists at all, and why baking both layers into one texture (the obvious "just Photoshop it" idea) cannot reproduce the look. The second rasterization is irreducible.

The second *transform* is not. A companion VU1 renderer clips and transforms each shared vertex once, then issues two `XGKICK`s from the same transformed buffer: the opaque wall primitive with `ABE=0` and the per-vertex tint, then the window primitive with `ABE=1`, a constant color, and the window texture. Measured on hardware, the city pass fell from 11.74 ms to 8.33 ms under load — a 29 percent reduction, from deleting work that was never visible to the GS.

Four details cost a debug cycle each, and generalize to any multi-kick renderer:

The per-kick texture switch is an **A+D settings block**, not a bind. Copy the texture's own settings block — its GIF tag plus `TEXFLUSH`, `CLAMP`, `TEX1`, `TEX0`, `TEXA` and both `MIPTBP` registers — wholesale, so punch-through `TEXA` keying and the custom mip pyramids of Chapter 21 ride along verbatim, and grow the `NLOOP` to append the blend register the kick needs.

Constant colors enter at **x128, not x255**. A textured primitive's modulate identity is 128, and the fork's own `GetMaxColorValue` says so; scaling by 255 renders at double brightness. This is the same trap that made untextured translucent geometry refuse to fade in Chapter 20, wearing a different hat.

A sentinel disables the second kick: writing `-1.0` into the window color slot turns the renderer into a wall-only path, so one renderer serves both the combined draw and any single-textured per-vertex-colored draw. Cheaper than two renderers.

Drain pending geometry construction while it owns its textures and admission state. Deferred construction reads these bindings; teardown first can construct old geometry under the next draw's state. `glFlush` commits construction here: it neither sends normal frame DMA nor waits for VU/GS completion. Referenced storage survives until that later boundary.

And afterwards, dirty ps2gl's blend cache: the window kicks program `ALPHA_1` behind the library's back, so the next blended draw must resend its own function or inherit an additive one.

What a clipper was quietly doing for you

Moving clipping to VU1 has a consequence that no one writes down. A software clipper **culls as a side effect**: a triangle wholly outside the frustum clips to zero vertices and is silently never emitted. Take the clipper out of the EE and that free rejection leaves with it. Every triangle in the buffer is now transformed, written to scratch, DMA'd across VIF1, and processed by VU1 — only to be discarded there.

The counters showed it plainly. After the double-kick landed, the VU1 path was submitting about ten thousand vertices per frame while the old two-pass EE path submitted about ten and a half thousand — nearly identical, when the new path should have been five thousand *lower*, because it no longer resubmits the wall stream. The missing five thousand were off-screen triangles that the EE clipper had been rejecting all along. The geometry streams span an entire build window; the camera sees a cone.

The remedy is cheap and belongs on the EE, because the eye coordinates it needs have already been computed by the transform: build a five-plane outcode per vertex, AND the three outcodes of a triangle, and drop the triangle when the result is non-zero — all three vertices outside the same plane means it can never be visible. For an outcode test derived from projected side bounds, a vertex behind the near plane contributes only the near bit. Applying those projected side tests behind the eye can reject a visible crossing triangle. This restriction does not replace correctly formulated homogeneous plane tests; preserve the conventions of the chosen representation.

Clipping is VU1's job. Deciding whether geometry is worth sending is still the EE's.

Discipline for the counters themselves

Choose aggregation windows to match the experiment. A fixed frame count spans different elapsed time at different frame rates; a fixed time window can span different scene content. Retain per-frame samples where possible and compare matched phases or routes. Isolate loading when measuring steady rendering, and include it explicitly when measuring transition latency.

Tag every line with the configuration it describes, and **restart the window when the configuration changes**. An A/B toggle that leaves the accumulator running silently blends two configurations into one average and produces a confident, meaningless number.

Adaptive LOD thresholds encode a measured cost curve. A faster submission path changes that curve. Recalibrate against the revised pipeline while preserving the visual policy; an old governor may keep removing affordable detail.

Chapter 24 - The Heap Smasher: Forensics When the Crash Site Lies

Relevant manuals: EE Core User's Manual Ch 4 (exception handling — reading `Cause`, `BadVAddr`, `EPC`); the newlib allocator source (`_mallocr.c`) for what a chunk header looks like on the ground.

Chapter 23's double-kick renderer shipped with a bug that no amount of frame measurement could see: a one-line buffer under-declaration that corrupted the heap on every wall-only draw and crashed the game minutes later, in a different subsystem, on a button press. This chapter is about how such a bug presents, why the obvious readings of the evidence were wrong twice, and the small toolkit that finally cornered it in three hardware runs.

The signature: an exception inside the allocator

The console showed an EE TLB load exception with `EPC` resolving (`addr2line`, always `addr2line` first) into `__malloc_update_mallinfo`, called from `_mallinfo_r`, with `BadVAddr 0x00000004`. Read that address the way the machine does: some chunk's `fd / bk` link was `NULL`, and the walker dereferenced `NULL + 4`. The allocator was walking its free list and stepped on a chunk header full of zeros.

This is the single most important fact about the whole class: **the crash site is a reader of earlier damage, not evidence that the allocator wrote it**. The heap was corrupted earlier — possibly minutes earlier — by code that has long since returned. Debugging `mallinfo` is debugging the coroner. The crash moving between builds, or appearing "when I press START" (the debug HUD's RAM row called `mallinfo()`), is the same misdirection: the button did not break anything; it was merely the next reader.

The tool: the walker itself, turned into a validator

The function that crashes on a corrupt heap is, by that very property, a useful probe for this failure. `mallinfo()` walks allocator metadata and can fault on a bad link — so call it on purpose, everywhere, with one discipline: **print a label before the walk, and a second line after**. On a healthy heap you get `chk X / ok X` pairs; on a corrupt one the output ends at a `chk` whose `ok` never came, and that label names the phase. (The first version of this instrument printed only after the walk and was therefore silent on exactly the runs that mattered — an instrument must be proven able to fire before its silence means anything.)

Deployed as a ladder — first around the stage-load phases, then around the frame's draw passes, then around the branches inside the guilty function — it took three hardware runs to descend from "the stage-3 load kills it somewhere" to one call: the first wall-only draw through the x2 renderer, 24 vertices of land platform.

The false exit: padding that "fixed" it

Mid-hunt, 16 bytes of padding made the crash disappear. That established layout sensitivity, not a repair or an addressing-class proof. Padding can change the victim, detection or timing of relative overruns as well as stale pointer writes. Keep the suppression as localization evidence, then identify the

actual writer. Here the decisive evidence was the subsequent mismatch between the prefix array's capacity and its real transfer count.

The writer: a staging array sized from a comment

The x2 renderer stages a 23-quadword prefix block per buffer — window color, two 10-quadword texture-settings blocks (giftag + nine A+D registers each), and a prim giftag template — unpacked to VU memory with `packet.Add(Pfx, 23)`. The C++ member backing it was declared `uint128_t Pfx[21]`: sized from a header comment that described the settings blocks as 9 quadwords. Every draw wrote `Pfx[21]` and `Pfx[22]`; the wall-only branch's `memset(&Pfx[12], 0, 11 * 16)` wrote 32 bytes of zeros past the object. And the object lives on the heap (`new CCliPTriX2Renderer`), so those zeros landed on the next malloc chunk's header — the NULL `bk` the walker found. The fix was one character. The lesson is structural: when a size is asserted in three places — the array, the transfer call, and the microcode's reserved range — derive all three from one constant, and if you must pick one to trust, pick the transfer call you can grep, never the prose.

Rules this chapter adds

An allocator exception can expose earlier corruption: resolve `EPC` and `ra`, then distinguish the failing reader from the writer. `mallinfo()` is a useful metadata probe for this failure, not a complete heap verifier; label before it. A perturbation that suppresses a memory bug is a localization result. Custom renderers are heap objects — a member-array overrun is heap corruption. And scaffolding is scaffolding: the per-frame walk costs real EE time and leaves with the fix.

Chapter 25 - GS Context Reuse and Mipmap Page Locality

Relevant manuals: GS User's Manual pp. 47-49 (the two register contexts and `PRIM.CTXT`), Ch 8.3/8.5 (block arrangement tables — the source of the packed-mip offset magic), and the trilinear/LOD sections the mip fix leans on.

Two costs can masquerade as a fill bottleneck: repeated texture-state synchronization and poor locality between sampled mip levels. The GS's two register contexts can reduce the first; format-aware packing can reduce the second. These experiments distinguish the mechanisms without treating a historical frame-rate plateau as a universal performance result.

One compound kick: the second context is not decoration

The double-kick still paid a hidden tax: per-micro-buffer settings prefixes, each with a `TEXFLUSH` — about 206 per frame at dense load. `TEXFLUSH` stalls the GS until in-flight drawing completes, then invalidates the texture cache, so the profile `READ` as a fill wall while much of it was serialization bubbles. **A stall pattern scales with draw count; true fill scales with coverage** — count texture-path state events per frame before blaming pixels.

The fix is the GS's own design: two persistent register contexts exist exactly so intermixed primitives can switch state with a bit in `PRIM` instead of repeated register loads. Walls draw on context 1 (live GL state, `ATE` pinned off), additive windows on context 2 (window texture, `ALPHA_2`, `TEST_2` — programmed once per pass), inside `ONE XGKICK`: wall gtag `EOP=0`, window gtag `EOP=1` closes the packet, and the zero-vertex case still emits the empty window tag so the kick terminates (hardware-proven over a 2700-frame torture rotor). Worth ~2.1 ms of GS wait at matched wall density. Two traps are load-bearing: **context 2 is not yours alone** — `glClear` draws through a context-2 draw env every frame and stomps it, so the renderer re-arms per pass; and the wall pass rides `LIVE` context-1 state, so whatever the old prefixes used to force (blend, alpha test) must now be pinned explicitly.

Ground sampling: isolate depth, primitive shape and memory layout

The remaining dips tracked the city's ground. A debug layer kill-switch priced it instantly — land off, 60 fps, every time — and then four serial hardware experiments tried to keep the layer visible but cheaper: draw it after the city so hidden pixels Z-fail; make the roads write depth so the street corridor rejects the land beneath; reshape the primitives (32u slivers, 128u strips, giant fans); finally draw it with the depth unit fully off (`ZMSK=1 + ZTST=ALWAYS` — no Z read, no Z write). **All four were flat.** That flatness is the finding: depth outcome, draw order, primitive shape, and the whole Z unit eliminated, the cost follows the sampled-texture path in that experiment. These flat probes support a sampling-side hypothesis; they do not identify every VU/GIF/GS stall or prove a universal cost model.

The sampling-side suspect: the land texture's mip levels each lived in their own GS page, and a grazing-angle ground plane keeps every pixel BETWEEN two LOD levels, so trilinear fetches two page-separated texels per pixel — a per-pixel page walk worth ~1.4-2.0 ms, roughly ten times the raw fill cost of the same pixels. A mip-disable experiment jumped every immovable window to the ceiling — but note the confound: that removes the page reloads AND trilinear's inherent cost together, so it only localizes the

path. The stronger comparison was pack the whole 64x64 pyramid into ONE page (identical texels, identical filtering, only the layout changed) and the windows locked at 60 with mips on. The layout improved that workload. It does not make trilinear free or prove every pyramid fits a page. Larger RGBA32 pyramids can require multiple pages and need separate layout and performance measurements.

The packed path has its own contract: magic block-offset tables from the GS manual's block arrangement tables (PSMT8 packs L0-L3 in one page; PSMCT16 packs L1-L3 beside a full L0 page), MIPTBP TBW=1 for the sub-64 levels while the upload keeps DBW=2, one locked memory-area owner per pack released exactly once, no lazy re-allocation allowed to retarget TEXO away from the pack — and scope discipline: only the exact tested pyramid shape takes the packed path; everything else stays on the proven legacy allocator.

Negative results and their limits

Front-to-back submission (no early-Z to feed — sorting cost with zero GS gain, measured worse than build order). Draw-order/Z-fail tricks on a fetch-bound surface (Z-fail saves only the write RMW, and that slice measured ~zero here). Strip-shape tuning (the column-count telemetry correlated beautifully with the dips and was a confound — the A/B is the judge, never the correlation). And a process rule that made the whole hunt converge in seven hardware slots over two days: one change per slot, expected log signature stated before the run, matched route, and any visual regression rejects the slot even when the numbers are flat.

Rules this chapter adds

- Compatible two-material draws can use both GS contexts in one compound kick. Account for other context users: the tested ps2gl `glClear` rewrites context 2, so the custom pass must restore it.
- Count TEXFLUSHes before blaming fill. Stalls scale with draws; fill with coverage.
- Improve mip locality within a proven format/owner layout. Not every pyramid fits a page; locality alone proves no complete-frame speedup.
- The flat matrix localizes invisible GS costs: vary depth outcome, order, shape, and the Z unit one slot at a time. Flat probes support a sampling hypothesis for that workload, not exclusive ownership of every stall.
- A layer kill-switch measures the effect of omitting that layer in matched windows. Overlap and cache changes prevent treating the delta as its isolated cost or a guaranteed bound on another optimization.
- The tested front-to-back candidate regressed. GS has no modern early-Z justification for sorting; preserve order and measure any proposed change.
- One change per hardware slot, signature declared before the run, and a visual regression rejects the slot regardless of the numbers.

Chapter 26 - Submission LOD, Texture Identity, and Semantic Vertex Channels

A performance radius must remove the second material

Distance-fading a material to alpha zero is a visual policy, not a submission optimization. The EE still transforms the vertices, DMA still transports them, the VU/GS still receives the primitive, and texture/blend work still exists. When the purpose of a radius is performance, the material must disappear from the command stream.

For example, keep an opaque facade shell to the main visibility limit while restricting an emissive overlay to a shorter distance. Intersect visible base spans with admitted overlay ranges, then submit wall-only gaps and compound intersections. The wall-only path must emit no overlay primitives. A transition may fade the overlay near admission, but excluded ranges should omit its submission entirely.

The reusable rule is simple: inspect the submitted kicks or primitives. If an "invisible" material still arrives at the GS, the LOD did not save its work.

Shared glTF images are one upload and many aliases

Several glTF materials may reference one `cgltf_image`. Uploading from each material independently creates pixel-identical GS residents. When the loader does not apply distinct sampler objects, source-image identity is the complete upload identity: search earlier materials for the same image, reuse their `Texture2D` handle, and decode/upload only on the first occurrence.

That turns the material texture slots into aliases. Deep model teardown must deduplicate by runtime texture id, release each handle once, and clear every alias before destroying the material arrays. In short, source-image identity controls upload and handle identity controls unload; filenames control neither.

Test reset/teardown and sampler differences explicitly. Identical source images do not justify sharing when conversion or sampler requirements produce different runtime resources.

Each shared vertex lane gets one semantic owner

Custom VU1 paths often repurpose a convenient input component, but the public renderer contract must state what that component means. For example, input wall alpha may carry a fog keep coefficient packed into `XYZF2.F`. That lane then represents fog, not ordinary wall opacity.

If the material specification calls for an unfogged emissive overlay, its kick must be independent of both that alpha lane and wall fog enable. It uses a constant contribution through `ALPHA.FIX`, and its giftag clears `PRIM.FGE`. Letting the second material inherit wall alpha or FGE would silently couple haze policy to window brightness and can tint additive content with the fog color. Another material may intentionally share fog; make that choice explicit.

Treat position fog, material opacity, transition alpha and additive strength as separate meanings even if an implementation temporarily packs one into a shared word. Every material kick opts into only the channels it owns.

Translate a global mip distance into signed S7.4 LODK

GS `TEX1.K` is logarithmic signed S7.4 bias. A global distance multiplier must preserve each texture's authored baseline by adding:

```
delta_K = -16 * log2(distance_scale)
```

Round once, add it to the asset's existing `LODK`, clamp to the signed 12-bit range, and touch only textures with mip levels. A scale below 1 selects coarser mips sooner/closer; above 1 holds finer levels farther away. Do not replace all asset K values with one global constant: ocean, facades and other families retain different aliasing requirements, and S7.4 quantization makes nearby UI values hardware-identical anyway.

Chapter 27 - Split Immutable Models at the Dynamic Material Boundary

ps2gl display lists capture geometry, texture and material state when they are compiled. A live tint cannot be pushed through an immutable whole-model list without either recompiling it or silently drawing the old captured color.

Install the model by mutability instead. Compile the stable body once, identify the smallest tintable material/submesh once at load time, and retain that piece as an explicitly parameterized draw.

HyperSolar's F22 uses this split for a single 32x32 grayscale exhaust mask: shared code precomputes static/live hue and brightness variants, while the PS2 path applies the cached tint to the small exhaust draw without searching the model or rebuilding its body list.

The general rule is: immutable work stays immutable; dynamic material islands stay small and explicit. In this example only 16 unique exhaust vertices need runtime lighting before the cached tint is applied; the body list is reusable. ps2gl already normalizes runtime color to the GS scale, so another blanket 128/255 correction was not the solution.

The first attempt required `mesh.colors`, but the loaded glTF primitive had no `COLOR_0` and the PS2 loader allocated no color array; the effect vanished. Draw-local colors fixed that assumption. A later attempt reset shared frame scratch and damaged pending skybox geometry. The repair appends normally through the existing copied draw path, as Chapter 9 explains. Validate the dynamic piece, the unchanged body and scratch lifetime separately. Correct tint alone does not establish complete lighting parity or performance.

Chapter 28 - Audio Ring Geometry Is a Wire ABI

An IOP streamer's chunk size simultaneously defines its encoded-file stride, DMA read size, SPU2 half-ring size, total per-channel ring size, latency and legal address range. Tuning one constant in isolation can overwrite a neighbor, replay a stale half or alias channel storage even when the individual transfer looks valid.

For a concrete layout, choose 1024 PS-ADPCM blocks per chunk. At 16 bytes per block each channel half is 16 KiB and its two-half ring is 32 KiB. Two rings can occupy `0x1F0000` and `0x1F8000` if the SPU2 allocator reserves those ranges. These are example addresses, not mandatory streaming locations. Encoder stride, IOP transfer length, ring addresses, loop flags and allocation map must agree. Derive shared constants where possible, audit every boundary after a change, and validate playback and refill deadlines on the target.

Deploy the format, not just the executable

The chunk size is also a contract with files already on removable media. Raw PS-ADPCM has no header declaring the interleave. A 2048-block channel chunk read by a 1024-block streamer is split between the two outputs: left receives the first passage, right receives the following passage. At 28 samples per block and 48 kHz, the offset is $1024 * 28 / 48000 = 0.597333$ seconds. It sounds like the song was started twice even though both voices are keyed together.

A real occurrence of this symptom disappeared when removable-media files were replaced with music encoded using the revised chunk geometry. Rebuilding the executable could not replace those files. Check deployed bytes and encoder settings before changing voice-start scheduling: a format mismatch can sound like a synchronization bug.

Build dependencies also matter. If a make prerequisite list expands a track variable before that variable is defined, the normal build can omit music even though a later target includes it. Attach dependencies after their inputs are defined, and verify generation and deployment as separate steps.

Halving a 2048-block chunk to this 1024-block example reduces stereo ring storage from 128 KiB to 64 KiB but also halves refill headroom. Correct playback of a track does not establish worst-case refill margin. Measure under the intended storage latency and IOP workload before adopting a smaller ring as a general memory optimization.

Chapter 29 - What Each Kind of Validation Can Prove

A PS2 renderer crosses compiler, cache, DMA, VU and GS boundaries. A test that observes one boundary cannot certify all the others. State what was tested and preserve the executable, settings and input that produced the result; a later source tree is a different candidate.

A host numerical model can check a matrix formula, packed address calculation or capacity bound. It cannot establish that the compiler preserved an aliasing trick, the EE published dirty cache lines, or the GS rasterized two clipped planes identically. Conversely, one correct image cannot prove a capacity bound or rule out a delayed source-lifetime failure.

Build an evidence chain

Use the cheapest relevant check first, then test the boundaries it cannot represent. For a new VU primitive, that might mean:

1. A host model for input/output bounds and attribute interpolation.
2. Inspection of the generated instructions and packet layout.
3. A standalone checkerboard test with near/side crossings and full buffers.
4. Integration with real state transitions and asynchronous source lifetimes.
5. Hardware comparison in representative views and display modes.

A successful build establishes toolchain integration, not rendering correctness. An emulator can expose exceptions and packet errors while hiding display timing or texture-precision behavior. Real hardware remains necessary, but its result is still scoped to the workload and configuration exercised.

Keep different claims separate

Correct pixels, lower EE preparation time, earlier GPU completion and steadier presentation are different outcomes. A change can improve one and regress another. Identify the expected effect before measuring, and reject any claim that relies on a counter whose population or coverage changed.

Long runs test failure modes short captures miss: origin drift, exhausted counters, stale descriptors, reset ownership and slow leaks. They complement focused boundary tests rather than replace them. Keep failed hypotheses with their evidence so that a plausible but already-disproved explanation does not become the next debugging plan.

The practical rule is simple: each result should say which implementation ran, what it observed, and what remains outside that observation. No project milestone or average frame rate substitutes for that information.

Chapter 30 - Construction, Publication, and Completion

Relevant manuals: EE User's Manual for DMAC/GIF/VIF; EE Core User's Manual for cache visibility; VU User's Manual for VU memory and PATH1 consumption.

A deferred draw has three separate boundaries. Geometry must be constructed while its renderer and material state are valid. Packets and external REF sources must be published with the required cache writeback. Referenced bytes must then stay immutable until the consuming transfer or processor completes. A state flush may establish only the first boundary; elapsed time establishes none of them.

Drain the old draw before tearing down its state

If a renderer reads live qualification flags when it constructs a packet, clearing those flags first can route old geometry through the wrong program. Drain pending construction under the old bindings before changing renderer or ending the material scope. Know whether an API named "flush" constructs work, submits it, or waits for it; those operations are not interchangeable.

Append-only frame scratch is a simple way to preserve pending references. Resetting a cursor inside a material helper can overwrite an earlier draw. Saving and restoring the cursor does not restore overwritten bytes. Reserve all storage before publishing a transactional draw; if a prefix has already escaped, a fallback must continue after it instead of replaying it.

Retained caches need publication identity as well as pointer identity. A frequently rebuilt array can be safely borrowed if it stays immutable through consumption. A static pointer is unsafe if its contents change while DMA still owns them. Flushing the command packet does not publish dirty external REF payloads automatically.

Presentation is another ownership transfer

Rendering completion, a serviced vblank and scanout ownership are distinct. A queued vblank event proves an edge occurred, not that blanking is active when the CPU later handles it. A GS SIGNAL marker and raster FINISH have different semantics. Design the swap policy around the actual completion event and display ownership, not the names of wrapper functions.

Double buffering is a storage arrangement, not a completion proof. Reuse a buffer only after its previous consumers have retired. The same rule applies to an EE/IOP RPC: a timeout may report failed progress, but it cannot permit overwriting a live request or issuing another transaction into the same client.

For queued IOP services, distinguish admission from execution. A reply that acknowledges an owned command copy can release the sender's source; it does not necessarily mean the worker has executed the command. Make both meanings explicit in the protocol.

Chapter 31 - Compact Geometry Without Changing Its Meaning

Compact inputs can reduce EE memory traffic and VIF transfer work. They are useful only if the consumer reconstructs the required geometry, attributes, draw order and lifetime. A screen-space rectangle, clipped world quad and perspective decal are not interchangeable simply because each has four corners.

Preserve the original triangle diagonal. For example, a strip ordered 0,1,3,2 corresponds to triangles 013/132; substituting the other diagonal can change color interpolation, texture gradients and coplanar depth. Share a center or basis only when the producer's arithmetic and clipping contract permit that reconstruction.

Near/side clipping must interpolate the consumer's actual UV, color and fog meanings. Unsupported inputs need an explicit fallback before any partial submission escapes. A center/radius rejection is not proof that a visible corner can be dropped or that a long beam qualifies as a small billboard.

Share preparation, not incompatible state

Stable parent transforms or visibility facts may serve several materials. That does not make their draws mergeable: texture, depth, blend, clipping and source lifetime must agree. Preserve the required base/overlay order even when a single preparation loop creates both outputs.

A specialized ordinary path and a cold exceptional adapter can share a VU program and bounded staging. Admit only the proven numerical domain; do not weaken a depth tolerance or discard rare geometry to increase fast-path use. Measure fallback frequency as well as the cost of the common path.

For HUD batching, retain logical order, glyph placement, unsupported-character advance and empty-string behavior. Keep animation clocks and RNG calls with their update owner. Capacity drains should end at complete prefixes, and descriptor sources must survive until consumed. Sorting overlapping UI by texture is not equivalent to batching adjacent compatible draws.

Bound coordinates in the frame where they are used

A periodic or origin-rebased world can keep player coordinates small while a retained render origin drifts far from the camera. Subtracting the same period from both does not reduce their difference. What matters to the transform is the size of the operands and their relative displacement.

Recenter retained local data as one publication transaction: update complete dependent spans in an inactive bank, preserve absolute simulation identities, and invalidate caches tied to the old origin. Publish only once all consumers agree on the new frame. This limits cancellation and fixed-point range problems without moving the represented world. Test boundary crossings and long runs; a short stationary image cannot exercise accumulated origin drift.

Chapter 32 - Emitted Code and Cache Dependencies

Relevant manuals: EE Core User's Manual and Instruction Set Manual for generated EE code; VU User's Manual for scheduled execution and hazards.

Source expressions, byte models and successful assembly are different witnesses. None alone proves what optimized EE code or a scheduled VU program will execute.

Packet writes need a valid language representation

In one GIF-tag bug, an optimized packet advertised fifteen A+D records while sending two: a type-punning store intended to change the count disappeared under optimization. Casting storage to an unrelated struct pointer does not establish a valid object of that type.

Use typed packet operations or representation copies supported by the language. Then compare tag counts with the actual DIRECT payload and inspect the generated code where necessary. A host model that manually writes the intended bytes cannot reproduce a store the compiler removed.

A source simplification can also introduce helper calls, spills or byte copies. Inspect emitted EE code before assuming fewer expressions mean fewer cycles. Specialize a proven finite input domain where useful, but retain out-of-domain behavior, signed zero where relevant, arithmetic order and wide capacity checks.

Inspect the final VU image

Check branch targets, delay slots, integer/vector dependencies, Q/ACC timing, store/load ordering and store-to-XGKICK hazards after scheduling. Include decoder relocation and the combined instruction/data-memory budget. A compiler scheduling barrier is not necessarily a physical hardware fence.

Test clipped output expansion and buffer transitions as well as ordinary vertices. A standalone program can validate the mechanism without reproducing integrated material admission, source lifetime or complete-frame cost. Numerical agreement and a performance improvement require separate evidence.

Cache keys describe dependencies

A sparse VU context writer must preserve the consumer's addresses and invalidate on unknown state, program changes and resets. GS environment dirtiness and retained VU context are separate. A transform-only unlit proof does not establish correctness for normal or light data.

A combined-matrix cache, for example, depends on projection, modelview and the GS raster scale. Key every relevant input or use a generation scheme that tracks all of them. Bit-identical reuse is a stronger guarantee than an epsilon comparison that silently changes output.

Retained texture-slot iterators can remove searches if eviction order, validity and ownership survive splices, deletion and reset. Measure changes independently to distinguish their contributions.

Chapter 33 - Mip Layout, Dither, and Depth

Relevant manuals: GS User's Manual for local-memory block geometry, TEX1, MIPTBP, CLAMP, TEST and dithering; GS Supplement for texture-page-buffer behavior. Address locality, sampled texels and complete-frame cost are different properties.

Isolate atlas cells at every level

A correct base atlas can bleed at smaller mip levels. Preserve each material's coverage or energy filter and exclude neighboring cells from every sampling footprint. Check sampler strides, resident ownership and teardown too.

A custom PATH1 renderer can emit a cell-specific CLAMP with each compact record, avoiding an EE material split for every cell. Preserve source STQ, draw order and the actual sampler bounds. Compare the extra GS state traffic against the EE work saved; fewer draw calls alone prove no gain.

Check packed block addresses and strides for the chosen PSM. Not every pyramid fits one page. Reduced address scatter alone proves no speedup: compare completion cost at matched camera views and workloads.

Zero-alpha blending can still write a different RGB16 value

A blend equation may return the stored destination RGB for source alpha zero. Writing that value through RGB16 dithering can nevertheless subtract another quantization step. Avoid the unwanted color store without accidentally suppressing an intended depth update.

For an admitted source-alpha blend, an emitted alpha test can reject zero alpha and use ZB_ONLY on failure to preserve the original depth behavior. A KEEP failure action is a different optimization: it also discards depth, so it needs depth writes disabled and a compatible destination-alpha contract. Explicit alpha tests, destination-alpha tests and other blend equations must retain their own semantics.

Copy temporary TEST settings into packet-owned storage before restoring logical state. Test transparent texels in the actual framebuffer format, with dithering enabled and disabled; qualify every admitted blend mode.

Work in the active raster units

Layer ordering and decal tolerance solve different problems. In a Z16 buffer, small relative depth tiers can quantize to the same value. Identical world planes can also produce different raster planes after clipping and XY/Z quantization. Use the active projection, raster scale and depth format when calculating a tolerance; preserve perspective STQ and decide which surface owns depth for subsequent draws.

If several layers intentionally form one background, composing their color before publishing the background depth can avoid unnecessary coplanar tests. That is a pass-order contract, not permission to disable depth globally or offset geometry until an artifact disappears.

Chapter 34 - Alignment Is Part of the Asset Format

Relevant manuals: EE Core Instruction Set Manual for memory-access alignment; EE User's Manual for DMA transfer units and source addresses.

An embedded payload is not aligned merely because its containing symbol is aligned. The address a consumer uses is the sum of the container base and every enclosing offset. The format must preserve alignment at each boundary.

For a consumer requiring alignment A, check:

```
(container_address + payload_offset + element_offset) % A == 0
```

Use the requirement of the actual operation. Ordinary typed loads and DMA transfers need not have the same alignment. A layout safe for byte parsing can fault when reused as an in-place int16 array; a CPU-readable buffer can still violate a qword DMA contract.

Padding belongs in the producer and the format

Round each payload start up to its required boundary, initialize the padding and record the padded offset in the format. Align the final embedded object as well. Use checked arithmetic for offset-plus-length calculations and reject out-of-range or misaligned sections before exposing typed pointers.

Validate the absolute address when the loader accepts arbitrary caller-owned buffers. Relative section checks alone are sufficient only if the outer buffer's alignment is already guaranteed. Where a format deliberately permits unaligned data, decode through byte-safe operations into aligned storage rather than casting it to a stricter type.

A HyperSolar asset incident illustrates the boundary: internal section offsets were aligned, but a changed content size placed the entire appended section at an odd address. In-place halfword loads on the EE faulted. Padding the outer section and validating its recorded offset repaired the contract; relying on the previous content size being even had never been safe.

Repacking and embedding must agree

A repacker must distinguish the original payload from a previously appended section; otherwise repeated runs can accumulate padding or duplicate data. Keep version, sizes and offsets together and validate them before reuse.

The generated binary and any generated C tables describing it are one ABI. Make the consumer depend on the producer's real inputs and completion output. Missing sibling outputs or a stale stamp must not leave a newly linked ELF using mismatched metadata. A successful link proves symbols resolved, not that the bytes embedded in them match their parser.

For DMA-fed texture data, apply alignment to every blob, including those after size words or other small fields. A single alignment directive at the start of an assembly file does not align all subsequent objects.

Chapter 35 - Profile the Pipeline Without Measuring the Instrument

Relevant manuals: EE Core User's Manual for counters and exceptions; EE User's Manual for timers, DMA and interrupt behavior. Counter semantics matter as much as counter resolution.

Start with the phase you intend to optimize

A PC histogram that includes loading can make a one-time builder look like a per-frame hotspot. Reset or re-arm collection at the intended phase and label transitions. Resolve samples against the matching unstripped ELF, including inline information when available.

Check bucket boundaries before assigning cost to a function. A 64-byte bucket, for example, can straddle a function entry and charge work to its neighbor. Use instruction/line information and callers to interpret it, rather than choosing an optimization from the hottest symbol label alone.

Window averages can hide sustained dense segments. Add bounded per-frame records around missed budgets, using the same phase definitions as normal reports. Keep nested scopes nested: their times are not independent amounts to add together. Independent maxima likewise do not describe one real frame.

Separate preparation, completion and presentation

EE time inside a draw call includes packet construction and any waits reached there. It is not a direct VU or GS execution timer. Measure completion and queue-ready boundaries separately, then correlate them with presentation. If the frame is queued promptly but finishes late, that narrows the problem; it does not by itself distinguish VU execution, GIF transfer and GS work.

Keep camera route, geometry, layers, display mode and instrumentation fixed for an isolated comparison. Changing content density changes the workload even on the same route. Short comparisons can test a bounded candidate; long runs address drift and rare failures. Neither a high mean FPS nor zero repeats in one run establishes a universal frame-time ceiling.

Telemetry has ownership and cost

Formatting and output can perturb the frame being measured. Capture compact records into bounded owned storage, then format or decode away from the hot path where practical. Keep schema identity and argument widths explicit when moving decoding to the host.

An asynchronous EE/IOP transport must retain request/reply storage until the RPC completes, distinguish queue admission from output completion, and report backpressure. A UDP send accepted by the local stack does not prove host receipt. Sequence numbers and drop counters help distinguish missing data from absent work.

Use configurable collection scopes and record which ones were enabled. Missing records from a disabled collector mean unknown, not zero. Include collector and reporting cost in whole-frame comparisons, and verify a repair without the timing perturbation that first made the problem disappear.

Appendix A - Quick Rules

- Treat EE, IOP, VU and GS addresses, caches and completion events as separate contracts.
- Standalone IOP bring-up and network-loader bring-up need different reset policies; do not destroy services still owned by the loader.
- Retain asynchronous RPC storage until the previous transaction is complete. A timeout does not release it.
- Keep game state and scene composition outside low-level backend helpers.
- Give display registers one owner; do not layer independent display libraries.
- Size framebuffer and Z reservations using their PSM page geometry. Validate every color/Z pairing and scanout mode on hardware.
- A 448-line progressive buffer can simplify page alignment; 480-line output is also possible with a correctly padded reservation and display setup.
- Match DISPLAY magnification to scan timing. Control BGCOLOR deliberately.
- Vblank notification, rendering completion and scanout ownership are distinct.
- Batch adjacent compatible draws without changing visible order.
- Use stock and custom renderer paths only within their attribute/state contract.
- A construction flush is not necessarily submission or hardware completion.
- Publish DMA sources with the required cache operations, then keep them immutable until all consumers retire. Buffer parity alone proves nothing.
- Preserve the original triangle diagonal and clipping interpolation when changing primitive representation.
- Admit fast paths transactionally; never hide capacity failures by dropping valid geometry.
- Configure overlay depth explicitly. Equal world planes do not guarantee equal raster depth; calculate decal tolerance in the active raster units.
- Keep any temporary depth tolerance out of later scene depth unless that is the intended ownership policy.
- When bypassing library color conversion, use the GS scale appropriate to the operation. Textured MODULATE identity is 128.
- Alpha-zero blending can still re-dither RGB16. Color suppression must preserve intended depth and destination-alpha behavior.
- Keep UV magnitudes bounded and preserve perspective STQ through clipping.
- Choose texture format and dimensions for the real resident budget. Track compatible slot sizes and CLUT demand, not only total free pages.
- Align each embedded DMA payload, not just the container's first symbol.
- Store MIPTBP with its texture state; prove level residency and reconstruct pyramids after a layout wipe.
- Count legacy mip allocations or verify a shared packed owner's full address layout, including CLUTs and teardown.
- Preserve atlas isolation at every mip level. Address locality alone is not a measured frame-time improvement.
- Treat vertex opacity, fog and secondary-material strength as distinct meanings.
- Set all fog-enabling state and submit the expected F coefficient. Account for its screen-space interpolation when selecting geometry and the fog law.
- Drain every handled GS interrupt source and acknowledge it correctly.

- Initialize renderer-selection data completely and handle unsupported matches.
- Check actual emitted packet counts, types and payload sizes.
- Size VU staging from real transfers and prove worst-case clipped output.
- Inspect scheduled VU instructions, relocation and memory budgets after changes. Compiler barriers and hardware hazard fences are different.
- Account for VCL's flag and alias-analysis limitations; do not assume a source expression preserves the intended MAC flags or Q lifetime.
- $v_{foo.w}$ is one; it is not the additive base for a zero-based w constant.
- Make variable-output loops terminate safely and emit only well-formed GIF packets, including deliberate empty completion tags.
- Match texture pixel lifetime to the library's copy/borrow/ownership API.
- Install immutable display lists once; keep dynamic material islands explicit.
- Resolve crash addresses against the matching ELF before diagnosing them.
- Allocator faults can expose earlier corruption. Padding that hides a fault localizes layout sensitivity; it does not identify or repair the writer.
- Compare reset-cycle memory at equivalent drained boundaries and state which pools are measured.
- Reserve SPU2 voices by role; identify reusable playback handles by generation.
- At 48 kHz, SPU2 pitch 0x1000 is unity. Match encoding rate, block flags and pitch policy.
- Keep encoder chunk geometry, DMA transfers, ring halves and deployed files consistent.
- Avoid making rendering wait for music refill or diagnostic output.
- Match filesystem paths to the loaded IOP driver stack; mass0: and legacy mass: are different interfaces.
- Test standalone boot separately from ps2link and test display/texture behavior on real hardware as well as an emulator.
- Attribute EE preparation, transfer/completion and presentation separately.
- Collect the intended phase, record enabled scopes and inspect sampling buckets.
- Missing telemetry is not zero work, and independent maxima are not one frame.
- Preserve evidence for a specific implementation and workload; a successful build, image, average or soak each establishes a different result.

Appendix B - Case Study: HyperSolar's PS2 Port

HyperSolar is an on-rails shooter used here as an example, not as a reference architecture every PS2 game should copy. Its port illustrates how ordinary rendering features expose the hardware boundaries discussed in this book.

The initial port combined shared scene logic with a small PS2 backend. Embedded images, GS-scaled alpha and perspective textured quads brought up the title scenes. Static models used display lists; rapidly changing effects used explicit draw helpers. This separated immutable geometry from live material parameters without duplicating gameplay on the new platform.

HUD rendering exposed the cost of repeated state setup. Grouping text and rectangles reduced it, while later colored-array paths required careful preservation of triangle diagonals, logical order and deferred source lifetime. The lesson was to batch compatible work, not to sort all UI by texture.

Dense city rendering exposed different bottlenecks. Texture residency required per-texture CLUT and mip state, with format-specific packed owners where useful. VU clipping needed complete output-capacity bounds. Reusing wall transforms for two materials reduced preparation, but each material still needed its own blend, fog and depth semantics.

Several apparent rendering defects were ownership defects. An overwritten scratch region damaged an earlier queued draw. A drifting retained origin lost precision despite bounded player coordinates. An aligned inner asset section faulted because its outer container offset was odd. Each repair strengthened a contract rather than depending on slower code or a lucky memory layout.

Audio demonstrated the same principle across processors: voices required playback identity, encoded files had to match the IOP ring geometry, and asynchronous command admission had to be distinguished from execution. Memory-card persistence and network-loader behavior needed separate tests.

These examples share a method: identify the producer and consumer, preserve the meaning and lifetime of their data, and measure the boundary actually being changed. Their particular scene layout, asset settings and performance results are not universal PS2 requirements.